

PROJEKTOVANJE SLOŽENIH DIGITALNIH SISTEMA
FUNKCIONALNA VERIFIKACIJA HARDVERA

**Hardverski akcelerator za prepoznavanje
šahovskih figura zasnovan na normalizovanoj
unakrsnoj korelaciji**

Grupa: y25-g10

Sadržaj

1. Uvod

- 1.1. Predmet
- 1.2. Polazna tačka: ESL model sistema
- 1.3. Ciljna platforma i korišćeni alati

2. Opis algoritma

- 2.1. Matematička definicija
- 2.2. Optimizacija 1: kvadrirana korelacija (eliminacija korena)
- 2.3. Optimizacija 2: integralna slika (SAT)
- 2.4. Optimizacija 3: egzaktna celobrojna Q1.31 podela
- 2.5. Bitska analiza širina signala
- 2.6. Promenljiva veličina šablona

3. Uklanjanje petlji

- 3.1. Izgradnja integralne slike
- 3.2. Srednja vrednost šablona
- 3.3. Glavna petlja

4. Interfejs i okruženje

- 4.1. Četiri RT interfejsa
- 4.2. Registarska mapa
- 4.3. Portovi RTL jezgra
- 4.4. Memorijsko okruženje

5. ASMD dijagram

- 5.1. Pregled stanja
- 5.2. Faza A: učitavanje slike i izgradnja SAT
- 5.3. Faza B: šablon i njegova srednja vrednost
- 5.4. Faza C: priprema prozora
- 5.5. Faza D: pipeline-ovana MAC petlja
- 5.6. Faza E: normalizacija i upis rezultata

6. Datapath i controlpath

- 6.1. Registri
- 6.2. Funkcionalne jedinice
- 6.3. Sekvencijalni delilac
- 6.4. Memorije
- 6.5. Blok dijagram

7. Analiza posle sinteze i implementacije

- 7.1. Metod
- 7.2. Utrošeni resursi

7.3. Kritična putanja i maksimalna frekvencija

7.4. Latencija i propusnost

7.5. Funkcionalna verifikacija

8. Pakovanje u IP jezgro

8.1. Izbor obrasca: slave sa internim memorijama

8.2. Dva AXI interfejsa

8.3. Regioni unutar memorijskog interfejsa

8.4. Kontrolni i statusni registri

9. Integracija u sistem

9.1. Topologija

9.2. Adresna mapa

9.3. Tok podataka po jednom polju table

9.4. Dva pravila koja softver mora poštovati

10. Analiza integrisanog sistema

10.1. Utrošeni resursi

10.2. Vremenska analiza i radni takt

10.3. Propusnost sistema

10.4. Poređenje sa referentnim rešenjem

11. Zaključak

12. Funkcionalna verifikacija

12.1. Uvod i nivoi verifikacije

12.2. Granice dopuštenog stimulusa

12.3. Kontrolni registri (AXI4-Lite, S00)

12.4. Memorijski prostor (AXI4-Full, S01)

12.5. Struktura verifikacionog okruženja

12.6. Zahtev na drajver izveden iz izvornog koda

12.7. Matrica testova

12.8. Plan pokrivenosti

12.9. Realizovano okruženje

12.10. Rezultati regresije (Vivado XSim 2025.2)

12.11. Funkcionalna pokrivenost

12.12. Dvostruka simulacija: Vivado XSim i Cadence Xcelium

12.13. Nalazi verifikacije

12.14. Zaključak verifikacije

1. Uvod

1.1. Predmet

Projekat realizuje hardverski akcelerator za prepoznavanje šahovskih figura sa fotografije table. Slika table dimenzija 720×720 piksela deli se na 64 polja dimenzija 90×90 piksela; za svako neprazno polje računa se normalizovana unakrsna korelacija (engl. *Normalized Cross-Correlation*, NCC) sa šablonima figura, a iz mape skorova se rekonstruiše pozicija u standardnoj Forsyth-Edwards (FEN) notaciji. Korelacija je, prema profilisanju izvršne specifikacije, uzročnik preko 70 % procesorskog vremena i time prirodan kandidat za akceleraciju u programabilnoj logici.

Dokument prati put od algoritma do implementiranog sistema, u tri celine. Prva (poglavlja 2–6) opisuje algoritam i RT model: matematičku osnovu i tri optimizacije, uklanjanje petlji, interfejs, dijagram algoritamske mašine stanja i podelu na datapath i controlpath. Druga (poglavlje 7) daje analizu samostalnog jezgra posle sinteze i implementacije. Treća (poglavlja 8–10) opisuje put do sistema na ploči: pakovanje jezgra u IP blok sa AXI interfejsom, integraciju sa procesorskim sistemom i DMA kontrolerom, i analizu celog integrisanog dizajna.

1.2. Polazna tačka: ESL model sistema

Sistem je prethodno modelovan na sistemskom nivou (ESL) u okruženju SystemC/TLM-2.0 i dokumentovan u okviru predmeta *Projektovanje elektronskih uređaja na sistemskom nivou*. Taj model definiše particionisanje hardvera i softvera, adresnu mapu, registarski interfejs akceleratora i vremenski budžet, i u ovom projektu se koristi kao **izvršna specifikacija**: RTL model mora dati bit-identičan rezultat.

Iz ESL modela se preuzimaju:

- bitske širine svih signala u datapath-u (ESL dokumentacija, Tabela 2), realizovane u odeljku 2.5;
- registarska mapa i adresni prostor programabilne logike (odeljak 4.2).

1.3. Ciljna platforma i korišćeni alati

Ciljna platforma je razvojna ploča **Digilent Zybo** sa uređajem **xc7z010clg400-1** (Zynq-7000). Uređaj raspolaže sa 17.600 LUT, 35.200 flip-floпова, 80 DSP48E1 blokova i 60 blokova RAMB36. Ploča nosi **512 MB** DDR3 memorije.

Nominalni takt programabilne logike je 100 MHz (period 10 ns) i samostalno jezgro ga ispunjava (odeljak 7.3). U integrisanom sistemu takt je **90,909 MHz**, jer PLL procesorskog sistema za traženih 95 MHz daje 1000/11 — odstupanje od nominalnih 100 MHz je 9,1 % (odeljak 10.2).

RT model je opisan u jeziku **VHDL-2008** i razvijen ručno iz ASMD dijagrama, ne kroz HLS. Simulacija je izvršena alatom **XSim**, a sinteza i vremenska analiza alatom **Vivado 2025.2**.

2. Opis algoritma

Akcelerator prima segment slike dimenzija do 90×90 piksela i šablon figure dimenzija do 30×30 piksela, i za **svaku** poziciju prozora (u, v) računa kvadrat normalizovane unakrsne korelacije. Rezultat je mapa skorova dimenzija (img_w - tmp_w + 1) × (img_h - tmp_h + 1), koju softver zatim pretražuje tražeći maksimum.

2.1. Matematička definicija

Koeficijent korelacije γ u tački (u, v), za segment slike f i šablon t , definisan je izrazom:

$$\gamma(u,v) = \frac{\sum_{x,y} [f(x,y) - \bar{f}_{u,v}] \cdot [t(x-u, y-v) - \bar{t}]}{\sqrt{\sum_{x,y} [f(x,y) - \bar{f}_{u,v}]^2 \cdot \sum_{x,y} [t(x-u, y-v) - \bar{t}]^2}}$$

gde je $f(x,y)$ intenzitet piksela segmenta, $\bar{f}_{u,v}$ srednja vrednost piksela onog dela slike koji se trenutno preklapa sa šablonom, t piksel šablona, a $\bar{t}$ srednja vrednost celog šablona. Normalizacija oduzimanjem srednjih vrednosti čini meru neosetljivom na promenu osvetljenja, što je i razlog za izbor NCC-a umesto obične korelacije.

Direktna implementacija ovog izraza je za hardver neprihvatljivo skupa: sadrži kvadratni koren, deljenje po svakoj poziciji prozora i ponovno sumiranje prozora u svakoj tački. Sledeća tri odeljka opisuju tri transformacije kojima se izraz svodi na oblik pogodan za realizaciju u programabilnoj logici, bez gubitka tačnosti.

2.2. Optimizacija 1: kvadrirana korelacija (eliminacija korena)

Rezultat korelacije se ne koristi kao brojna vrednost, već isključivo za poređenje sa pragom detekcije: figura je prepoznata ako je $\gamma > 0,75$. Pošto je $\gamma \in [-1, 1]$ i pošto se traži maksimum po apsolutnoj vrednosti, poređenje se ekvivalentno može izvesti nad **kvadratom** korelacije:

$$\gamma^2(u,v) > 0,75^2 = 0,5625$$

Time kvadratni koren potpuno nestaje iz datapath-a, a izraz koji hardver računa postaje:

$$NCC^2(u,v) = \frac{(\sum \text{diff_f} \cdot \text{diff_t})^2}{\sum \text{diff_f}^2 \cdot \sum \text{diff_t}^2}$$

gde su $\text{diff_f} = f(x,y) - \bar{f}$ i $\text{diff_t} = t(x,y) - \bar{t}$. Umesto korena i jednog deljenja po tački, ostaju tri akumulacije i jedno deljenje.

2.3. Optimizacija 2: integralna slika (SAT)

Srednja vrednost prozora $\bar{f}_{u,v}$ zavisi od pozicije prozora, pa bi je naivna implementacija računala iznova za svaku poziciju — to je $\text{tmp_w} \times \text{tmp_h}$ sabiranja po **svakoj** od nekoliko hiljada pozicija. Umesto toga, jednom po segmentu se gradi **integralna slika** (engl. *summed-area table*, SAT), u kojoj svaki element sadrži sumu svih piksela iznad i levo od sebe:

$$\text{SAT}[y+1][x+1] = \text{SAT}[y][x+1] + \text{row_sum}(y, 0..x)$$

Suma proizvoljnog pravougaonog prozora tada se dobija u konstantnom vremenu, sa četiri pristupa memoriji:

$$\text{sum_f}(u,v) = \text{SAT}[v+th][u+tw] - \text{SAT}[v][u+tw] - \text{SAT}[v+th][u] + \text{SAT}[v][u]$$

Složenost pripreme je $O(\text{img_w} \times \text{img_h})$ i plaća se jednom po segmentu, dok se trošak po prozoru svodi sa $O(\text{tmp_w} \times \text{tmp_h})$ na $O(1)$. Za realan slučaj (segment 90×90 , šablon 30×30) to je razlika između 900 i 4 sabiranja po svakoj od 3.721 pozicija.

2.4. Optimizacija 3: egzaktna celobrojna Q1.31 podela

Rezultat po poziciji prozora je razlomak iz opsega $[0, 1]$, koji se prenosi kao 32-bitna neoznačena vrednost u formatu **Q1.31** (vrednost 1,0 odgovara heksadecimalnom $0x80000000$). ESL model je ovo računao u pokretnom zarezu, uz zaštitnu granicu `if (ncc2 > 1.0) ncc2 = 1.0` koja je bila potrebna zbog grešaka zaokruživanja tipa `double`.

RT model umesto toga izvodi **tačnu celobrojnu podelu** u proširenom registru:

$$NCC^2 = (\text{num_sq} \ll 31) / \text{den_prod}$$

gde su `num_sq` i `den_prod` 52-bitne veličine, pa se deljenik računa u 83 bita. Pošto se ne koristi aritmetika u pokretnom zarezu, greška zaokruživanja ne postoji, a Cauchy-Schwarz-ova nejednakost garantuje $\text{num_sq} \leq \text{den_prod}$ egzaktno — zaštitna granica iz ESL modela matematički otpada i nije implementirana. Degenerisani slučaj konstantnog prozora ili konstantnog šablona (nulta varijansa, deljenje nulom) obrađuje se eksplicitno: rezultat je tada 0.

2.5. Bitska analiza širina signala

Širine svih signala u datapath-u realizovane su u paketu `ncc_pkg.vhd`. Tabela 1 daje njihov pregled.

Veličina	Biti	Tip u <code>ncc_pkg.vhd</code>
piksel slike / šablona	8	<code>pixel_t</code> — unsigned(7 downto 0)
dimenzije <code>img_w/h</code> , <code>tmp_w/h</code>	8	<code>dim_t</code> — unsigned(7 downto 0)
$\bar{f}$, <code>template_mean</code>	8	<code>mean_t</code> — unsigned(7 downto 0)
<code>diff_f</code> , <code>diff_t</code>	9	<code>diff_t</code> — signed (8 downto 0)
<code>sum_num</code>	27	<code>numacc_t</code> — signed (26 downto 0)
<code>sum_den_f</code> , <code>sum_den_t</code>	26	<code>denacc_t</code> — unsigned(25 downto 0)
<code>num_sq</code> , <code>den_prod</code>	52	<code>sq52_t</code> — unsigned(51 downto 0)
rezultat (Q1.31)	32	<code>result_t</code> — unsigned(31 downto 0)
akumulator integralne slike	32	<code>sat_t</code> — unsigned(31 downto 0)

Tabela 1. Bitske širine signala datapath-a i njihova realizacija u VHDL-u

`diff_f`, `diff_t` i `sum_num` su namerno označeni tipovi: razlika piksela i srednje vrednosti je negativna kad god je piksel tamniji od proseka, a suma proizvoda tih razlika prati taj predznak. Zato ove veličine

moraju biti *signed*, ne *unsigned*; broj bita (9 i 27) ostaje određen samo opsegom vrednosti.

Ceo datapath je zbog toga celobrojan. Srednje vrednosti $\bar{f}$ i $\bar{t}$ se zaokružuju na ceo broj pre oduzimanja (deljenje sa zaokruživanjem: $(\text{suma} + \text{count}/2) / \text{count}$), pa su diff_f i diff_t egzaktni celi brojevi — kroz desetine hiljada sabiranja u akumulatorima ne ulazi nikakva greška zaokruživanja. Jedina veličina u formatu sa fiksnim zarezom je konačni rezultat.

2.6. Promenljiva veličina šablona

Dimenzije tmp_w i tmp_h su parametri koji se zadaju u vreme izvršavanja, preko komandnih registara, a ne konstante u vreme prevođenja. To je neophodno jer se šabloni figura razlikuju: top je 25×15 , dok su kraljica i kralj do 30×30 . Konstante $\text{MAX_IMG_W} = \text{MAX_IMG_H} = 90$ i $\text{MAX_TMP_W} = \text{MAX_TMP_H} = 30$ predstavljaju samo gornju granicu za dimenzionisanje memorija, a ne pretpostavku o fiksnoj veličini.

Ista fleksibilnost omogućava i grubi prolaz *coarse-to-fine* algoritma na sistemskom nivou (segment 45×45 , šablon 15×15), pri čemu samo jezgro ne mora ništa da zna o toj strategiji — ono obrađuje dimenzije koje mu se zadaju.

3. Uklanjanje petlji

ASMD dijagram ne poznaje konstrukte `for` i `while`: svaka petlja mora biti prevedena u oblik `if-goto`, u kome brojači petlji postaju registri, a telo petlje jedno ili više stanja. U algoritmu postoje tri mesta sa petljama i ukupno četiri brojača.

3.1. Izgradnja integralne slike

Pre transformacije, dvostruka `for` petlja:

```
for (y = 0; y < img_h; y++) {
    row_sum = 0;
    for (x = 0; x < img_w; x++) {
        row_sum = row_sum + image[y*img_w + x];
        SAT[(y+1)*(img_w+1) + (x+1)] = SAT[y*(img_w+1) + (x+1)] + row_sum;
    }
}
```

Posle transformacije, dva brojača (`y` i `x`) postaju registri, a telo petlje stanja s oznakom `if-goto`:

```
    y = 0
L1: if (y >= img_h) goto L1_END
    row_sum = 0
    x = 0
    L2: if (x >= img_w) goto L2_END
        row_sum = row_sum + image[y*img_w + x]
        SAT[(y+1)*(img_w+1) + (x+1)] = SAT[y*(img_w+1) + (x+1)] + row_sum
        x = x + 1
        goto L2
    L2_END:
        y = y + 1
        goto L1
L1_END:
```

3.2. Srednja vrednost šablona

Pre transformacije:

```
sum_t = 0;
for (p = 0; p < tmp_w*tmp_h; p++)
    sum_t = sum_t + templ[p];
template_mean = (sum_t + count/2) / count;
```

Posle transformacije, jedan brojač `p` kao linearni indeks kroz šablon:

```
    p = 0; sum_t = 0
L3: if (p >= tmp_w*tmp_h) goto L3_END
    sum_t = sum_t + templ[p]
    p = p + 1
    goto L3
L3_END:
    template_mean = (sum_t + count/2) / count
```

3.3. Glavna petlja

Četiri ugnježdene nivoa: spoljni par (v, u) prolazi kroz pozicije prozora, unutrašnji par (y, x) kroz piksele šablona. Brojači y i x su isti registri kao u odeljku 3.1 — dve faze nisu istovremeno aktivne, pa se resurs deli.

Pre transformacije, četiri ugnježdene for petlje:

```
for (v = 0; v < res_h; v++) {
    for (u = 0; u < res_w; u++) {
        sum_f = SAT[(v+th)*(iw+1)+(u+tw)] - SAT[v*(iw+1)+(u+tw)]
            - SAT[(v+th)*(iw+1)+u] + SAT[v*(iw+1)+u];
        f_bar = (sum_f + count/2) / count;
        sum_num = 0; sum_den_f = 0; sum_den_t = 0;
        for (y = 0; y < tmp_h; y++) {
            for (x = 0; x < tmp_w; x++) {
                diff_f = image[(v+y)*img_w + (u+x)] - f_bar;
                diff_t = templ[y*tmp_w + x] - template_mean;
                sum_num += diff_f*diff_t;
                sum_den_f += diff_f*diff_f;
                sum_den_t += diff_t*diff_t;
            }
        }
        if (sum_den_f == 0 or sum_den_t == 0)
            result = 0;
        else
            result = (sum_num*sum_num << 31) / (sum_den_f*sum_den_t);
        result_map[v*res_w + u] = result;
    }
}
```

Posle transformacije:

```

    v = 0
L4: if (v >= res_h) goto L4_END
    u = 0
L5: if (u >= res_w) goto L5_END
    sum_f = SAT[(v+th)*(iw+1)+(u+tw)] - SAT[v*(iw+1)+(u+tw)]
        - SAT[(v+th)*(iw+1)+u]      + SAT[v*(iw+1)+u]
    f_bar = (sum_f + count/2) / count
    sum_num = 0; sum_den_f = 0; sum_den_t = 0
    y = 0
L6: if (y >= tmp_h) goto L6_END
    x = 0
L7: if (x >= tmp_w) goto L7_END
    diff_f = image[(v+y)*img_w + (u+x)] - f_bar
    diff_t = templ[y*tmp_w + x]        - template_mean
    sum_num  = sum_num  + diff_f*diff_t
    sum_den_f = sum_den_f + diff_f*diff_f
    sum_den_t = sum_den_t + diff_t*diff_t
    x = x + 1
    goto L7
L7_END:
    y = y + 1
    goto L6
L6_END:
    if (sum_den_f == 0 or sum_den_t == 0)
        result = 0
    else
        result = (sum_num*sum_num << 31) / (sum_den_f*sum_den_t)
    result_map[v*res_w + u] = result
    u = u + 1
    goto L5
L5_END:
    v = v + 1
    goto L4
L4_END:

```

4. Interfejs i okruženje

4.1. Četiri RT interfejsa

RT metodologija zahteva da se za svaki modul definišu četiri grupe signala: komandni, statusni, ulazni podaci i izlazni podaci. Tabela 2 daje njihovo konkretno značenje kod NCC jezgra.

Interfejs	Realizacija
Komandni	Konfiguracioni registri REG_IMG_W/H, REG_TMP_W/H, REG_IMG_ADDR, REG_TMP_ADDR (upisuju se pre pokretanja) i REG_CTRL (upis jedinice pokreće obradu). Na nivou RTL jezgra to su ulazi <code>img_w/h</code> , <code>tmp_w/h</code> i signal <code>start</code> .
Statusni	Registar REG_STATUS (0 = zauzet, 1 = završeno); na nivou jezgra izlazi <code>busy</code> i <code>done</code> .
Ulazni podaci	Ne prenose se preko porta kao vrednosti, već se čitaju iz memorije preko adresno-podatkovnog para: <code>img_addr_o / img_data_i</code> i <code>templ_addr_o / templ_data_i</code> .
Izlazni podaci	Mapa rezultata se upisuje u memoriju preko <code>result_addr_o</code> , <code>result_data_o</code> i <code>result_wr_o</code> ; procesor je čita sa offseta ADDR_RESULTS.

Tabela 2. Realizacija četiri RT interfejsa

Širina izlaznog porta izvedena je funkcionalno iz širina ulaza: pošto su `num_sq` i `den_prod` 52-bitni, a količnik posle skaliranja sa 2^{31} po Cauchy-Schwarz-u nikad ne prelazi 2^{31} , rezultat staje u 32 bita bez zasićenja.

4.2. Registarska mapa

Offset	Registar	Opis	Pristup
0x00	REG_IMG_W	širina segmenta slike	U
0x04	REG_IMG_H	visina segmenta slike	U
0x08	REG_TMP_W	širina šablona	U
0x0C	REG_TMP_H	visina šablona	U
0x10	REG_IMG_ADDR	adresa segmenta u deljenoj memoriji	U
0x14	REG_TMP_ADDR	adresa šablona u deljenoj memoriji	U
0x30	REG_CTRL	upis vrednosti 1 pokreće obradu	U
0x34	REG_STATUS	0 = zauzet, 1 = završeno	Č
0x40	ADDR_RESULTS	početak mape rezultata (32 bita po poziciji)	Č

Tabela 3. Registarska mapa NCC jezgra (preuzeta iz ESL modela)

Adresni prostor programabilne logike, definisan još u ESL modelu, je: deljena memorija na 0x4000_0000, prvo NCC jezgro na 0x5000_0000, drugo na 0x5100_0000 i DMA kontroler na 0x6000_0000. Mapa je od početka projektovana tako da odgovara rasporedu adresa opšte namene na Zynq platformi, pa se može preuzeti neizmenjena pri integraciji u blok dizajn.

Tabela iznad opisuje izvršnu specifikaciju, ne konačnu realizaciju. Tri bazne adrese jezgara i DMA kontrolera prenesene su **neizmenjene** (odeljak 9.2), ali dve stvari se razlikuju, jer je jezgro spakovano kao slave sa internim memorijama a ne kao master:

- deljene memorije na 0x4000_0000 u realizaciji **nema**, pa registri REG_IMG_ADDR i REG_TMP_ADDR ostaju **rezervisani** — nema adrese koju bi procesor javljao;
- REG_STATUS u realizaciji ima **dva odvojena bita** (završeno, zauzeto) umesto jedne vrednosti, jer se stanja *još nije počelo* i *u toku* inače ne mogu razlikovati.

Stvarni registarski interfejs, sa obrazloženjem oba odstupanja, dat je u odeljku 8.4; puna adresna mapa sistema u odeljku 9.2.

4.3. Portovi RTL jezgra

Entitet ncc_core ima interfejs prikazan u Tabeli 4. Adresno-podatkovni parovi ponašaju se kao sinhroni memorijski port: adresa se postavlja u jednom taktu, a podatak je validan u narednom.

Port	Smer	Tip	Značenje
clk, rst	in	std_logic	takt i asinhroni reset
start	in	std_logic	pokretanje obrade (jedan takt)
busy, done	out	std_logic	status obrade
img_w, img_h	in	dim_t	dimenzije segmenta
tmp_w, tmp_h	in	dim_t	dimenzije šablona
img_addr_o / img_data_i	out / in	integer / pixel_t	čitanje segmenta
templ_addr_o / templ_data_i	out / in	integer / pixel_t	čitanje šablona
result_addr_o, result_data_o, result_wr_o	out	integer / result_t / std_logic	upis mape rezultata

Tabela 4. Portovi entiteta ncc_core

4.4. Memorijsko okruženje

Jezgro ne poseduje memorije iz kojih čita: adresu izdaje, a podatak dobija u narednom taktu, kao kod svakog sinhronog memorijskog porta. Ono što se nalazi *sa druge strane* tog porta je pitanje okruženja, a ne jezgra — i upravo se to okruženje razlikuje između izvršne specifikacije i realizacije:

- **izvršna specifikacija** pretpostavlja *deljenu* memoriju programabilne logike, kojoj pristupa više mastera: procesor, DMA kontroler i oba jezgra;
- **realizacija** koristi obrazac *slave sa internim memorijama*: memorije su privatne za blok, dvoportne, i procesor ili DMA ih pune preko AXI interfejsa dok jezgro stoji. Deljene memorije u sistemu **nema**.

Za sam RT model to ne menja ništa — portovi jezgra su u oba slučaja identični, i zato je jezgro ušlo u IP blok nepromenjeno. Obrazloženje odluke i njene posledice na registarsku mapu su u odeljku 8.1.

Integralna slika je u oba slučaja izuzetak: ona je interna i privatna za jezgro, jer se u fazi izgradnje čita i piše u svakom taktu i ne bi tolerisala deljeni pristup.

5. ASMD dijagram

5.1. Pregled stanja

Kontrolna jedinica je Murova mašina stanja sa **23 stanja**, opisana u dvoprocesnom stilu: jedan proces sadrži isključivo registre (`signal_reg <= signal_next`), drugi svu kombinacionu logiku, sa podrazumevanim dodelama na početku i grananjem po `case state_reg`. Memorija integralne slike izdvojena je u treći, poseban proces. Stanja se prirodno grupišu u pet faza, a sva su — svih 23 — nabrojana u Tabeli 5 i prikazana na ASMD dijagramu (Slike 1a i 1b).

Faza	Stanja	Broj taktova	Namena
—	S_IDLE	1	čekanje na <code>start</code> , preuzimanje dimenzija, računanje <code>res_w/res_h</code>
A	S_LOAD_IMG_ADDR, S_LOAD_IMG_DATA	$2 \cdot \text{img_w} \cdot \text{img_h}$	čitanje segmenta piksel po piksel i istovremena izgradnja integralne slike
B	S_LOAD_TMPL_ADDR, S_LOAD_TMPL_DATA, S_CALC_MEAN, S_CALC_MEAN_WAIT	$2 \cdot \text{tmp_w} \cdot \text{tmp_h} + \sim 20$	čitanje šablona, sumiranje i deljenje radi dobijanja <code>template_mean</code>
C	S_L_V, S_L_U_A, S_L_U_B, S_L_U_C, S_L_U_D, S_L_U_E, S_L_U_WAIT	$6 + \sim 19$ po prozoru	četiri sekvencijalna čitanja SAT, računanje <code>sum_f</code> i deljenje radi <code>f_bar</code> ; nuliranje akumulatora
D	S_L_YX_FILL, S_L_YX_RUN, S_L_YX_DRAIN	$\text{tmp_w} \cdot \text{tmp_h} + 1$ po prozoru	pipeline-ovana akumulacija tri sume nad pikselima šablona
D	S_L_YX_DRAIN2	1	Akumulira poslednji registrovani par u sve tri sume (<code>sum_num</code> , <code>sum_den_f</code> , <code>sum_den_t</code>); poništava <code>mac_v</code> .
E	S_NCC_SQ	1	Provera <code>den = 0</code> ; ako nije, registruje <code>num_sq <= sum_num²</code> i <code>den_prod <= sum_den_f · sum_den_t</code> .
E	S_NCC_DIV, S_NCC_WAIT, S_WRITE_RESULT	~ 84 po prozoru	Startuje <code>div_ncc</code> nad registrovanim vrednostima: <code>(num_sq << 31) / den_prod</code> ; upis rezultata.
—	S_DONE	1	spuštanje <code>busy</code> , postavljanje <code>done</code>

Tabela 5. Svih 23 stanja kontrolne jedinice, grupisano po fazama

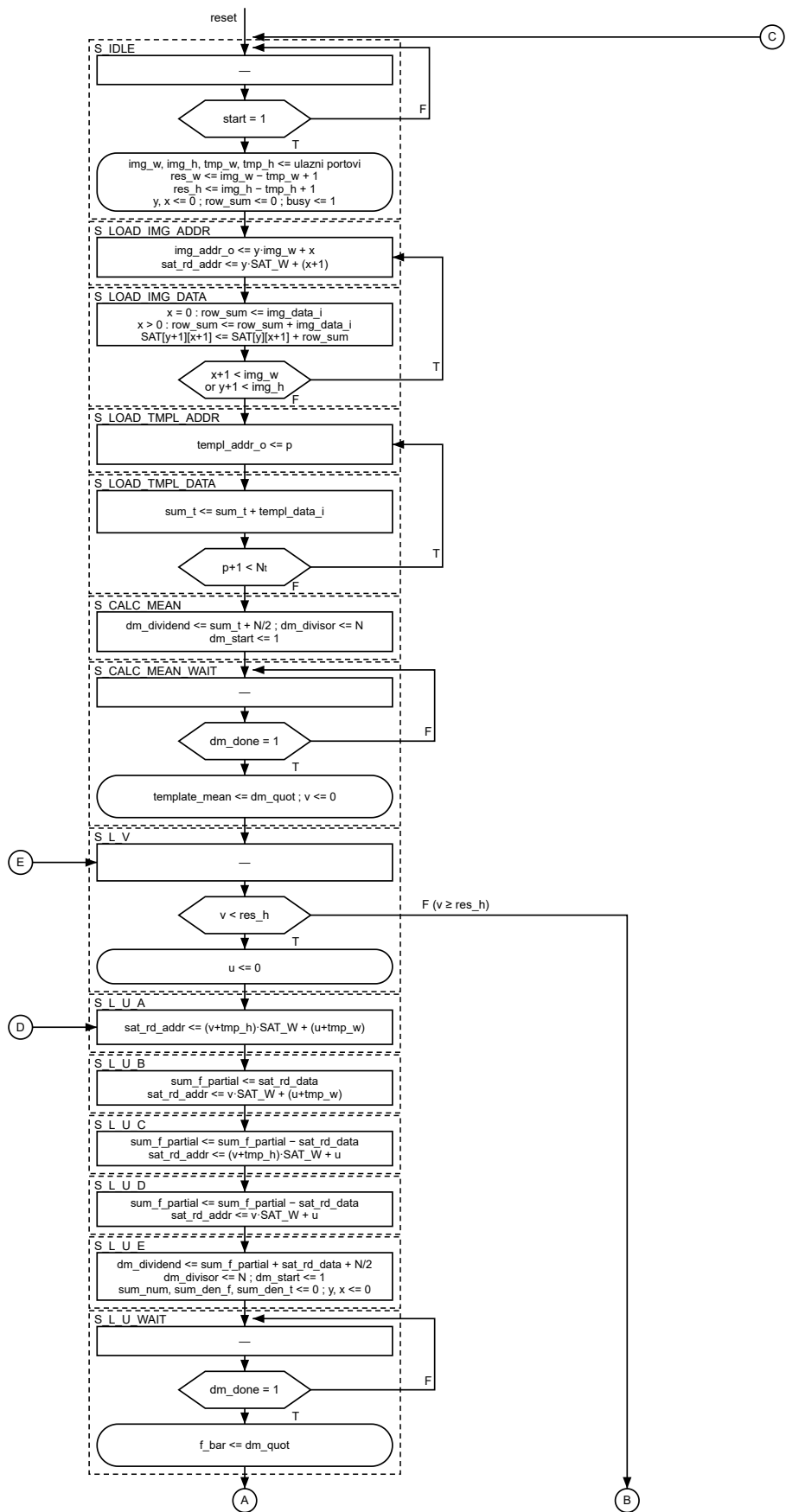
Slike 1a i 1b prikazuju ASMD dijagram, u notaciji sa vežbe RT modelovanja (Slika 4.5 udžbeničkih materijala). Zbog veličine je podeljen na dve slike, a pet grana koje ih povezuju obeleženo je krugovima A–E. Oblici imaju sledeće značenje:

- **isprekidana kontura** sa imenom stanja u gornjem levom uglu — jedan *ASMD blok*, odnosno jedno stanje kontrolnog automata, odnosno jedan takt;
- **pravougaonik** (*state blok*) — RT operacije koje se u tom stanju izvršavaju **bezuslovno**;
- **šestougao** — *kombinaciona odluka*; ne uvodi novo stanje, već se izračunava u istom taktu i bira sledeće stanje, sa granama T i F;

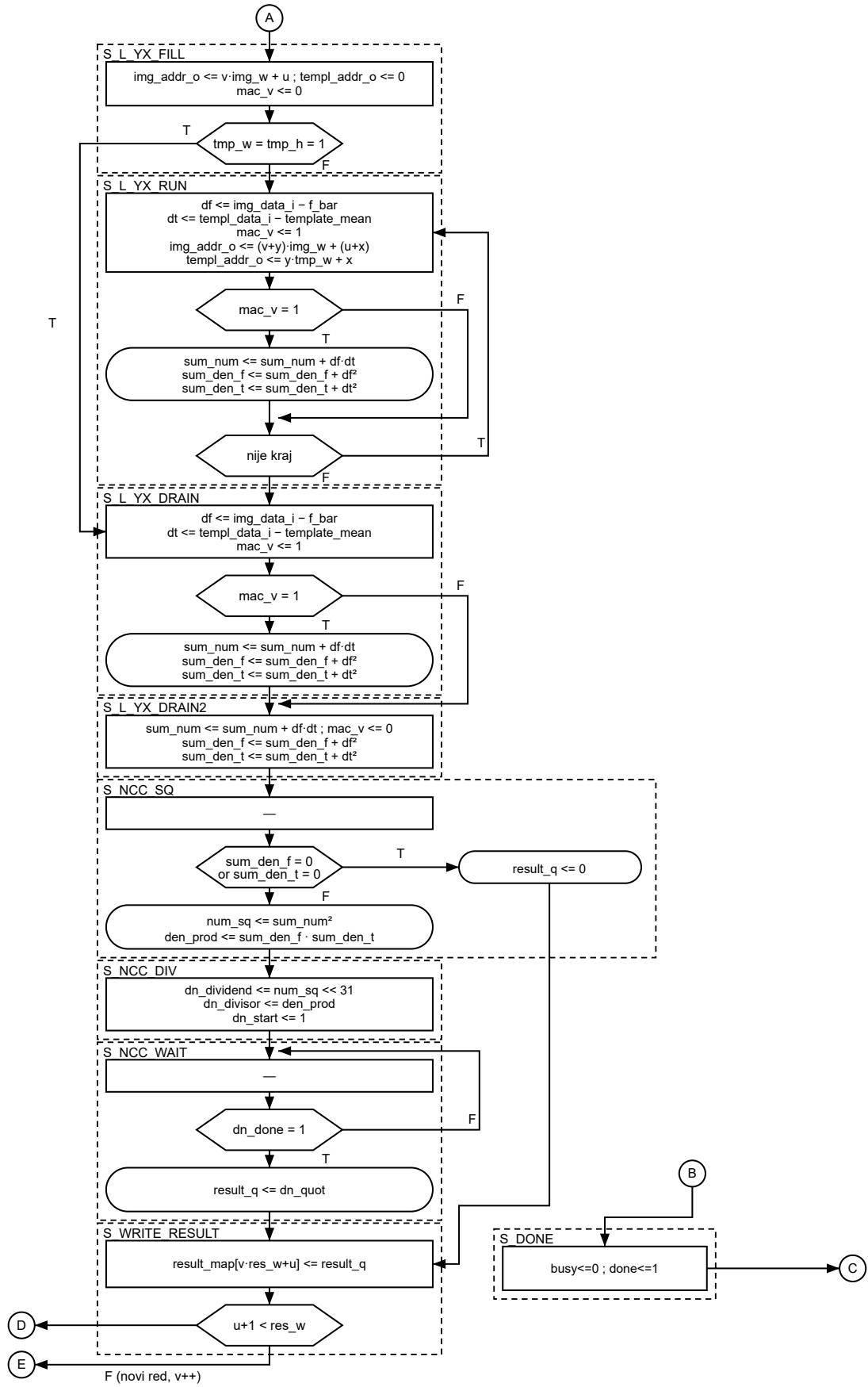
- **zaobljeni pravougaonik** (*uslovni izlazni blok*) — RT operacije koje se izvršavaju **samo na toj grani**. Tako se u stanjima čekanja vidi da se registar upisuje tek kad delilac javi da je gotov, a ne u svakom taktu čekanja.

Svaki ASM blok ima **tačno jedan** *state* blok — on i jeste stanje, i za njega se vezuje ime. Test i uslovni izlazni blokovi su opcioni. Pet stanja nema nijednu bezuslovnu RT operaciju — tri koja čekaju delilac (S_CALC_MEAN_WAIT, S_L_U_WAIT, S_NCC_WAIT), S_IDLE koje čeka start, i S_NCC_SQ gde obe grane vode u različite uslovne izlazne blokove. Kod njih je *state* blok **prazan** (crtica), što je tačan iskaz: u tom taktu se ne izvršava nijedna netrivialna RT operacija bezuslovno.

Trivijalne RT operacije oblika $\text{registar} \leftarrow \text{registar}$, koje postoje u svakom stanju, se po konvenciji iz vežbe ne crtaju; u HDL modelu ih pokrivaju podrazumevane dodele na početku kombinacionog procesa. Grupisanje u pet faza (A–E, Tabela 5, odeljci 5.2–5.6) je organizaciono, u tekstu — nije deo notacije same slike.



Slika 1a. ASMD dijagram, prvi deo — stanja S_IDLE do $S_L_U_WAIT$ (učitavanje segmenta i šablona, priprema prozora).
Krugovi A–E su veze sa Slikom 1b: A nastavak toka, B kraj obrade, C povratak u S_IDLE , D naredni prozor u redu, E novi red prozora.



Slika 1b. ASMD dijagram, drugi deo — stanja $S_L_YX_FILL$ do S_DONE (protočna MAC petlja, deljenje i upis rezultata).
 Krugovi A–E odgovaraju istoimenim krugovima na Slici 1a.

5.2. Faza A: učitavanje slike i izgradnja SAT

Segment se čita piksel po piksel, a integralna slika se gradi **u istom prolazu** — nije potrebna zasebna faza. Par stanja postoji zbog kašnjenja sinhronne memorije: u stanju `S_LOAD_IMG_ADDR` izdaju se adresa piksela i adresa elementa integralne slike iznad tekućeg, a u stanju `S_LOAD_IMG_DATA` oba podatka su validna, pa se računa nova vrednost `row_sum` i upisuje novi element integralne slike. Cena faze je dva takta po pikselu, odnosno 16.200 taktova za segment 90×90 .

5.3. Faza B: šablon i njegova srednja vrednost

Šablon se čita linearno, po istom obrascu adresa/podatak, uz akumulaciju sume. Po završetku se pokreće sekvencijalni delilac koji računa $\text{template_mean} = (\text{sum_t} + N/2) / N$, gde je $N = \text{tmp_w} \cdot \text{tmp_h}$. Deljenje sa zaokruživanjem je namerno: zaokružena celobrojna srednja vrednost je preduslov za bit-egzaktan datapath (odjeljak 2.5). Ova faza se izvršava jednom po pozivu.

5.4. Faza C: priprema prozora

Za svaku poziciju prozora treba pročitati četiri elementa integralne slike. Pošto je integralna slika realizovana kao blok-memorija sa jednim portom, četiri čitanja **ne mogu** biti istovremena — razložena su u pet stanja (`S_L_U_A` do `S_L_U_E`), pri čemu svako stanje izdaje sledeću adresu i istovremeno akumulira podatak koji je stigao po prethodnoj. U petom stanju se, uz poslednji sabirak, pokreće delilac za `f_bar` i nuliraju sva tri akumulatora, a `S_L_U_WAIT` čeka rezultat deljenja.

5.5. Faza D: pipeline-ovana MAC petlja

Ovo je unutrašnja petlja i ubedljivo najskuplji deo obrade: izvršava se $\text{tmp_w} \cdot \text{tmp_h}$ puta po svakoj poziciji prozora, dakle preko 3,3 miliona puta po pozivu za pun segment. Naivna realizacija bi po pikselu trošila dva takta (jedan za adresu, jedan za podatak). Umesto toga, faza je organizovana protočno, u **tri stepena**: u stanju `S_L_YX_RUN` se **istovremeno** izdaje adresa piksela $k + 2$, registruju razlike df/dt piksela $k + 1$, i akumulira već registrovani par piksela k (u `sum_num`, `sum_den_f`, `sum_den_t`). Stanje `S_L_YX_FILL` puni protočnu strukturu (izdaje samo prvu adresu, bez registrovanja i akumulacije), a `S_L_YX_DRAIN` i `S_L_YX_DRAIN2` su prazna za izdavanje adrese — samo dovršavaju registrovanje i akumulaciju piksela koji su već učitani. Slika 2 prikazuje preklapanje.

takt	t0	t1	t2	t3	...	tN-1	tN	tN+1
stanje	FILL	RUN (ponavlja se N-1 puta)				RUN	DRAIN	DRAIN2
izdaje adresu	px 0	px 1	px 2	px 3	...	px N-1	—	—
registruje df/dt	—	px 0	px 1	px 2	...	px N-2	px N-1	—
akumulira	—	—	px 0	px 1	...	px N-3	px N-2	px N-1

Ukupno: $N + 2$ takta za N piksela (jedan takt po pikselu + dva takta režije).

Bez protočnosti bi bilo $2N$ taktova — ušteda blizu 50 % ukupne latencije jezgra.

Slika 2. Trostepena protočnost unutrašnje petlje: preklapanje izdavanja adrese, registrovanja razlika df/dt i akumulacije

5.6. Faza E: normalizacija i upis rezultata

U stanju `S_NCC_SQ` se najpre proverava degenerisani slučaj: ako je bilo koja od suma kvadrata jednaka nuli, rezultat je 0 i deljenje se preskače. Inače se registruju $\text{num_sq} \leq \text{sum_num}^2$ i $\text{den_prod} \leq \text{sum_den_f} \cdot \text{sum_den_t}$. Stanje `S_NCC_DIV` pomera tako registrovan deljenik za 31 mesto ulevo i pokreće 83-bitni delilac nad registrovanim vrednostima. Stanje `S_NCC_WAIT` čeka njegov signal završetka, a `S_WRITE_RESULT` upisuje rezultat u mapu i pomera brojače prozora. Kvadriranje je razdvojeno od pokretanja delioca u dva takta u Koraku 8b: putanja od kvadriranja `sum_num` do 83-bitnog delioca bila je najgora posle rutiranja, pa je presečena registrom.

6. Datapath i controlpath

6.1. Registri

Registri su identifikovani iz oblika `if-goto` (poglavlje 3), grupisanjem po odredišnoj promenljivoj. Tabela 6 daje pregled.

Registar	Širina	Upisuje se u fazi
<code>img_w, img_h, tmp_w, tmp_h, res_w, res_h</code>	8	S_IDLE (na start)
<code>y, x</code> — deljeni: izgradnja SAT i unutrašnja petlja	7	A i D
<code>v, u</code> — pozicija prozora	7	C i E
<code>p</code> — linearni indeks šablona	10	B
<code>row_sum, sum_t, sum_f_partial</code>	32	A, B, C
<code>template_mean, f_bar</code>	8	B, C
<code>df_reg, dt_reg</code> — registrovane razlike piksel-sredina (MAC stepen 1, Korak 8b)	9	D
<code>mac_v_reg</code> — validnost stepena 1 (gejtuje akumulaciju u taktu punjenja)	1	D
<code>sum_num</code> (označen)	27	D (akumulator)
<code>sum_den_f, sum_den_t</code>	26	D (akumulatori)
<code>num_sq_reg, den_prod_reg</code> — registrovani ulazi 83-bitnog delioca (Korak 8b)	52	E
<code>result_q</code>	32	E
<code>busy, done</code>	1	S_IDLE, S_DONE

Tabela 6. Registri datapath-a

6.2. Funkcionalne jedinice

Datapath sadrži tri paralelna množača u unutrašnjoj petlji ($df_reg \cdot dt_reg$, df_reg^2 , dt_reg^2), koji se preslikavaju u DSP48E1 blokove, kao i dva množača van petlje (kvadriranje brojioca i proizvod imenilaca). Deljenje resursa nije primenjeno na tri množača unutrašnje petlje: pošto se ta petlja izvršava milionima puta, serijalizacija bi utrošila najskuplji deo obrade radi uštede od svega nekoliko DSP blokova, kojih ionako ima dovoljno (odjeljak 7.2).

Pet registara iz tabele iznad (`df_reg`, `dt_reg`, `mac_v_reg`, `num_sq_reg`, `den_prod_reg`) nisu deo prvobitnog dizajna — uvedeni su u Koraku 8b isključivo radi zatvaranja vremenskih uslova, bez ikakve promene izlaza. Njihova cena je **+0,41 %** latencije i **+110 flip-flopova**, a dobitak **2,59 ns** vremenske rezerve — zbir doprinosa protočnog stepena u MAC petlji i registra pred deliocem.

6.3. Sekvencijalni delilac

Deljenje promenljivim deliocem je najskuplja operacija u datapath-u. Realizovano je kao zaseban parametrizovan modul `seq_divider`, po klasičnom algoritmu sa vraćanjem (engl. *restoring division*): u svakom taktu se ostatak pomera za jedno mesto, poredi sa deliocem i po potrebi umanjuje, čime se dobija jedan bit količnika. Kritična putanja po taktu je time svedena na jedno oduzimanje i jedno poređenje.

Modul se instancira dva puta, sa različitim parametrom širine:

Instanca	Širina	Namena	Taktova po deljenju	Poziva po pozivu jezgra
div_mean	18	template_mean i f_bar	~19	1 + res_w·res_h
div_ncc	83	$NCC^2 = (\text{num_sq} \ll 31) / \text{den_prod}$	~83	res_w·res_h

Tabela 7. Instance sekvencijalnog delioca

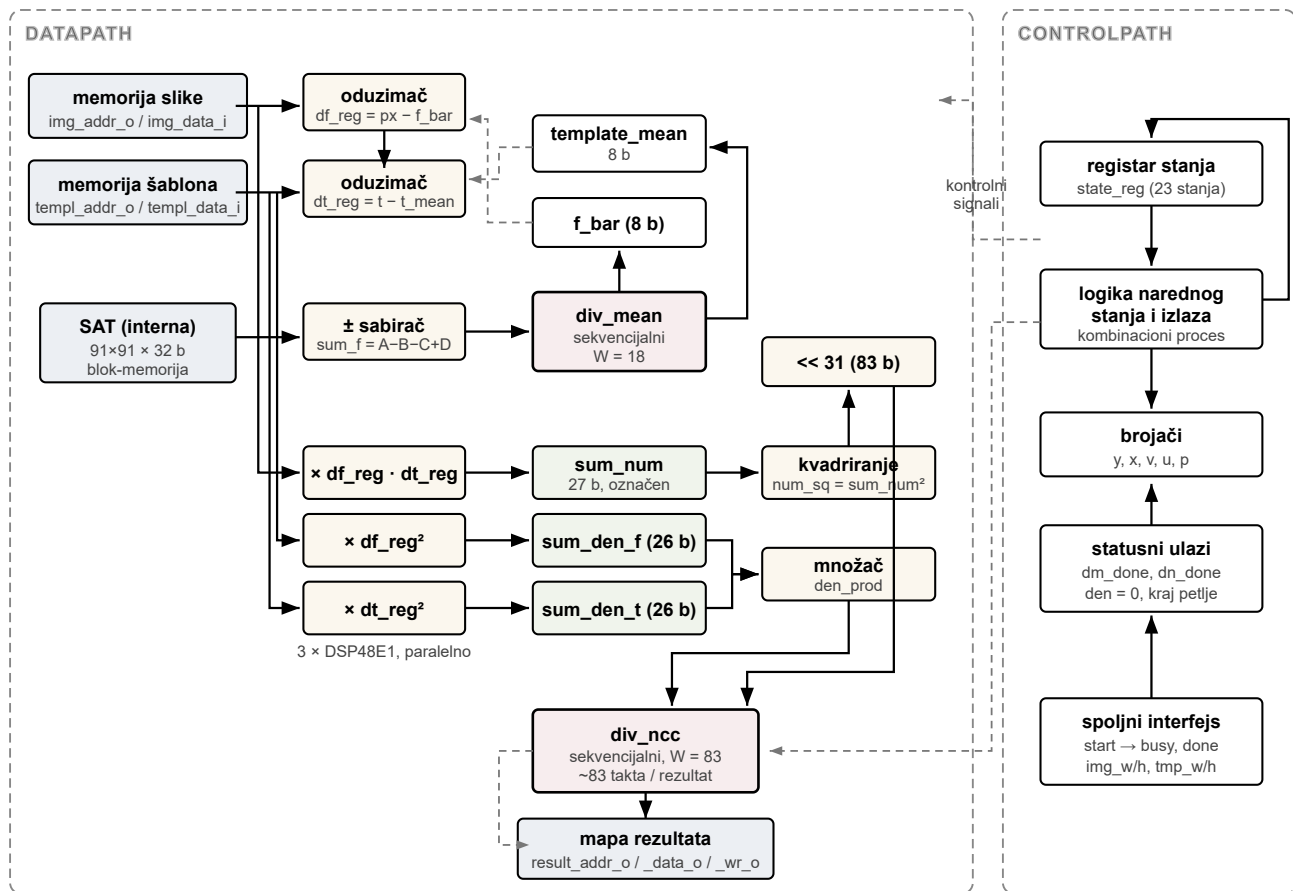
Instanca div_mean deljena je između dve namene (srednja vrednost šablona i srednja vrednost prozora) jer se te dve operacije nikada ne izvršavaju istovremeno — primer deljenja resursa koje ništa ne košta. Komunikacija sa deliocem je preko rukovanja: kontrolna jedinica postavlja operande i generiše impuls start, zatim čeka signal done u posebnom stanju čekanja.

6.4. Memorije

Memorija	Veličina	Pristup	Realizacija
segment slike	$90 \times 90 \times 8 \text{ b}$	čitanje	spoljna, deljena (procesor / DMA / oba jezgra)
šablon	$30 \times 30 \times 8 \text{ b}$	čitanje	ista spoljna memorija, druga adresna oblast
integralna slika (SAT)	$91 \times 91 \times 32 \text{ b}$	čitanje i upis	interna , privatna, blok-memorija
mapa rezultata	do $90 \times 90 \times 32 \text{ b}$	upis (jezgro), čitanje (procesor)	spoljna, offset ADDR_RESULTS

Tabela 8. Memorije u sistemu

6.5. Blok dijagram



Slika 3. Blok dijagram datapath-a i controlpath-a NCC jezgra (pune linije: tok podataka; isprekidane linije: upravljački i statusni signali)

Podela je klasična: controlpath je isključivo mašina stanja sa brojačima, koja prema datapath-u šalje upravljačke signale (izbor operanada, dozvole upisa, impulse za pokretanje delilaca), a od njega prima statusne signale (završetak deljenja, nulta varijansa, kraj petlji). Datapath ne sadrži nikakvu odluku o toku izvršavanja.

Radi preglednosti, slika ne iscertava zasebne blokove za pet protočnih registara iz Koraka 8b (df_reg/dt_reg/mac_v_reg između oduzimača i množača, num_sq_reg/den_prod_reg između kvadriranja i delioca) — oni takt-granicu unose unutar prikazanih putanja, bez promene redosleda blokova ni toka podataka.

7. Analiza posle sinteze i implementacije

7.1. Metod

Analizirano je **samostalno jezgro** (fajlovi `ncc_pkg.vhd` i `ncc_core.vhd`); ceo integrisani sistem obrađen je u poglavlju 10. Sinteza i implementacija su izvedene u alatu Vivado 2025.2, u režimu out-of-context za uređaj `xc7z010clg400-1`, uz pun tok do fizičke optimizacije posle rutiranja. **Sve brojke u ovom poglavlju su post-route.** Latencija je izmerena simulacijom u alatu XSim, brojanjem taktova u kojima je signal busy aktivan.

Ograničenje takta se čita pre sinteze, jer bi se inače jezgro sintetisalo bez vremenskog cilja. Režim out-of-context (OOC) sintetiše jezgro kao samostalan blok, bez povezivanja njegovih portova na stvarne pinove uređaja — alat tako ne pokušava da smesti I/O bafere za sve signale. To je ovde i nužno, jer golo jezgro ima 118 portova, a paket `clg400` nudi samo 100 korisničkih pinova.

7.2. Utrošeni resursi

Resurs	Utrošeno	Raspoloživo	Iskorišćenost
Slice LUT (sve kao logika)	1.472	17.600	8,36 %
Slice registri (FF)	664	35.200	1,89 %
DSP48E1	9	80	11,25 %
Block RAM (RAMB36E1)	9	60	15,00 %

Tabela 9. *Utrošeni resursi samostalnog jezgra posle implementacije (xc7z010clg400-1, post-route)*

Jezgro zauzima manje od devetine logičkih resursa uređaja.

U integrisanom sistemu jedna instanca nije golo jezgro nego ceo IP `ncc_accel` — jezgro sa dva AXI kontrolera i memorijskim podsistemom — što po instanci iznosi 1.887 LUT umesto 1.472. Zauzeće celog sistema je 35,4 % LUT-ova (odjeljak 10.1).

7.3. Kritična putanja i maksimalna frekvencija

Veličina	pri 11,000 ns	pri 10,000 ns
Zadata frekvencija	90,909 MHz	100 MHz
Najgora vremenska rezerva (WNS)	+0,387 ns	+0,146 ns
Kašnjenje kritične putanje	10,638 ns (logika 5,719 · rutiranje 4,919)	9,832 ns (logika 7,901 · rutiranje 1,931)
Broj nivoa logike	16 (9×CARRY4, LUT)	12 (10×CARRY4, 2×DSP48E1)
Krajnjih tačaka koje krše ograničenje	0 od 1.804	0
Maksimalna frekvencija		~101,5 MHz

Tabela 10. *Rezultati vremenske analize samostalnog jezgra (post-route, xc7z010clg400-1), pri dva zadata perioda takta*

Samostalno jezgro zatvara i ograničenje od 100 MHz, sa rezervom od 0,146 ns i ostvarivom frekvencijom od približno 101,5 MHz. Nijedna krajnja tačka ne krši ograničenje ni pri jednom od dva perioda.

Navedena su dva merenja zato što alat optimizuje samo do zadatog ograničenja: na 11 ns kritična putanja iznosi 10,638 ns, a na 10 ns svega 9,832 ns, jer je pri strožem cilju kvadriranje mapirano u DSP48E1. Reč je o istom RTL-u u dve različite implementacije, pa se rezerva ne sme aritmetički prevoditi između ograničenja.

Kritična putanja vodi od akumulatora `sum_num_reg` do registra `num_sq_reg`, kroz kvadriranje brojioca. Ona se izvršava jednom po poziciji prozora, a ne u unutrašnjoj petlji, pa je presek registrom na tom mestu koštao svega jedan takt po prozoru.

Rezultat je posledica odluke da se sva tri deljenja izvode sekvencijalno, jedan bit količnika po taktu. Kombinatorno deljenje promenljivim deliocem davalo bi rezervu od približno –46 ns, dakle kolo ne bi radilo ni na 20 MHz.

7.4. Latencija i propusnost

Latencija je izmerena na stvarnim podacima: segment 90×90 (polje a8 šahovske table) i šablon crnog topa 25×15, što daje mapu rezultata 66×76, odnosno 5.016 pozicija prozora.

Veličina	Vrednost
Latencija po pozivu (izmereno)	2.461.201 taktova
Broj obrađenih pozicija prozora	5.016
Taktova po piksel-operaciji	1,31
Vreme po pozivu pri 100 MHz	24,61 ms
→ propusnost	~203.800 pozicija/s (4,91 μs po poziciji)
Vreme po pozivu pri 90,909 MHz	27,07 ms
→ propusnost	~185.300 pozicija/s (5,40 μs po poziciji)

Tabela 11. Izmerena latencija i izvedena propusnost

Latencija u taktovima je svojstvo dizajna i ne zavisi od takta, pa su navedene obe frekvencije: 24,61 ms je ono što jezgro može samo, a 27,07 ms ono što integrisani sistem stvarno radi na 90,909 MHz (odeljak 10.3).

Izmerena latencija se poklapa sa analitičkim modelom. Trošak po poziciji prozora iznosi:

$$T_{prozor} = tmp_w \cdot tmp_h + 112 \text{ taktova}$$

Sabirak 112 čine čitanje integralne slike, deljenje pri računanju f_bar , punjenje i pražnjenje protočne strukture, registrovanje međurezultata, 83-bitno deljenje i upis rezultata. Za šablon 25×15 model predviđa 2.459.761 takt, a izmereno je 2.461.201 — odstupanje od **0,06 %**. Model se zato može koristiti za predviđanje latencije pri drugim veličinama zadatka.

Faza	Model	Taktova	Udeo
A — učitavanje slike i izgradnja SAT	$2 \cdot \text{img_w} \cdot \text{img_h} = 2 \cdot 90 \cdot 90$	16.200	0,7 %
B — učitavanje šablona i njegova srednja vrednost	$2 \cdot N + 19 \quad (N = 375)$	769	< 0,1 %
C, D, E — glavna petlja	$\text{res_w} \cdot \text{res_h} \cdot (N+112) = 66 \cdot 76 \cdot 487$	2.442.792	99,3 %
ostatak (prelazi S_L_V, zaokruženja modela)	—	1.440	0,1 %
Ukupno (izmereno)	—	2.461.201	100 %

Tabela 12. Raščlanjenje latencije po fazama (šablon 25×15)

Glavna petlja nosi 99,3 % vremena, što potvrđuje da je baš njenu protočnost trebalo najviše ubrzati.

7.5. Funkcionalna verifikacija

Ispravnost je proverena na dva nivoa — nad golim jezgrom i nad spakovanim IP-om kroz AXI. Svi testovi porede rezultat sa referentnim modelom u jeziku C, koji je nezavisno proveren nad celom tablom: 32 od 32 figure ispravno prepoznate.

Poređenje između dva nivoa je samo po sebi rezultat: IP daje **bit-identičan** izlaz kao golo jezgro, čime je pokazano da AXI omotač ne unosi izmenu u obradu.

Test	Podaci	Očekivano	Ishod
ncc_core_tb	sintetički segment 4×4, šablon 2×2	svih 9 pozicija prozora bit-identično referentnom modelu, uključujući oba ekstrema (0x00000000 i 0x80000000)	prolazi (9 / 9)
ncc_core_real_tb	stvarni segment 90×90 (polje a8) i šablon crnog topa 25×15	maksimum 0x80000000 na poziciji (u = 32, v = 14)	prolazi (0x80000000 @ 32,14)
ncc_accel_tb	isti stvarni podaci, ali upisani i pročitani kroz AXI (Lite za registre, Full za memorije)	isti maksimum, na linearnom indeksu 956 — dokaz da je omotač transparentan	prolazi (0x80000000 @ idx 956)
ncc_accel_burst_tb	AXI burst dužine 8 (awlen/arlen = 7), proveru WSTRB i ranog AW	svih 8 beat-ova ispravno u oba smera; upis sa isključenim bajt-lane-om ne menja piksel	prolazi (8 / 8 čitanje i upis)

Tabela 13. Rezultati funkcionalne verifikacije

8. Pakovanje u IP jezgro

8.1. Izbor obrasca: slave sa internim memorijama

Da bi jezgro postalo upotrebljivo u alatu za integraciju blokova, mora dobiti standardni interfejs. Od dva uobičajena obrasca — akcelerator kao **master** koji sam čita deljenu memoriju, ili kao **slave** sa sopstvenim internim memorijama — izabran je drugi.

Master interfejs bi zahtevao novu mašinu stanja za adresiranje i rukovanje odgovorima. Slave obrazac ne dira jezgro: `ncc_core` ulazi u omotač nepromenjen. Cena je što se segment slike i šablon moraju preneti u interne memorije pre svakog poziva — u izmerenom radu na ploči to iznosi oko 10,5 % ukupnog vremena (odjeljak 10.3).

Time se odstupa od sistemskog modela, koji pretpostavlja deljenu blok-memoriju na adresi `0x4000_0000`. U slave obrascu te memorije nema, pa registri `REG_IMG_ADDR` i `REG_TMP_ADDR` postaju rezervisani — zadržani su u mapi da adrese ostalih registara ostanu nepromenjene.

8.2. Dva AXI interfejsa

Blok izlaže dva odvojena slave interfejsa, jer se saobraćaj deli na nekoliko kratkih pristupa registrima i na masovni prenos piksela.

Interfejs	Tip	Širina adrese	Opseg	Uloga
S00_AXI	AXI4-Lite	6 bita	4 KB	kontrolni i statusni registri
S01_AXI	AXI4-Full	17 bita	128 KB	interne memorije: slika, šablon, rezultat

Tabela 14. AXI interfejsi IP bloka `ncc_accel`

Za registre je AXI4-Lite dovoljan. Za memorije je uzet pun AXI4 zbog *burst* prenosa — bez njih bi DMA izdavao po jednu transakciju na svaku reč. Sam IP i interkonekt podržavaju burst (dokazano testom `ncc_accel_burst_tb`, dužina 8), ali se u trenutnoj aplikaciji na ploči ne koriste: DMA je isključen (odjeljak 10.3), pa prenose radi procesor, reč po reč.

Širina adrese na interfejsu `S01` mora biti tačno 17 bita, jer je toliko širok port ka memorijskom podsistemu. Čarobnjak za pakovanje ume tu vrednost tiho da vrati na podrazumevanu (10 bita), pa je u kod dodata provera koja zaustavi sintezu ako širina nije 17 — bez nje bi IP samo prekrrio dva od tri memorijska regiona, bez ijedne greške pri prevođenju.

8.3. Regioni unutar memorijskog interfejsa

Sve tri memorije dele isti AXI interfejs i razlikuju se po gornja dva bita adrese. Dekodovanje je čisto kombinaciono: `region = addr(16:15)`, indeks reči = `addr(14:2)`.

Region	<code>addr(16:15)</code>	<code>offset</code>	Sadržaj	Organizacija
Slika	00	+0x00000	pikseli segmenta	8 bita × 8.192
Šablon	01	+0x08000	pikseli šablona	8 bita × 1.024
Rezultat	10	+0x10000	NCC ² u formatu Q1.31	32 bita × 8.192

Tabela 15. Regioni unutar `S01_AXI` (offset od bazne adrese bloka)

Jedan piksel zauzima celu 32-bitnu reč, iako mu je dovoljan bajt. Time procesor i DMA pristupaju svim regionima na isti način, bez pakovanja bajtova. Cena je četiri puta veći prenos kroz magistralu, ali prenos nije usko grlo (odjeljak 10.3). Regioni su poravnati na 32 KB, a najduži burst je 1 KB, pa nijedan burst ne može preći granicu regiona.

8.4. Kontrolni i statusni registri

Bazne adrese registara zadržane su identične konstantama sistemskog modela, tako da softver iz prethodne faze projekta radi bez menjanja adresa.

offset	Registar	Pristup	Značenje
0x00	IMG_W	upis/čitanje	širina segmenta slike
0x04	IMG_H	upis/čitanje	visina segmenta slike
0x08	TMP_W	upis/čitanje	širina šablona
0x0C	TMP_H	upis/čitanje	visina šablona
0x10	IMG_ADDR	—	rezervisan (videti 8.1)
0x14	TMP_ADDR	—	rezervisan (videti 8.1)
0x30	CTRL	upis	upis vrednosti sa bitom 0 = 1 pokreće obradu
0x34	STATUS	čitanje	bit 0 = done (pamti se), bit 1 = busy

Tabela 16. *Registri dostupni preko S00_AXI*

CTRL je impuls, ne stanje — jedan upis pokreće jednu obradu; da je stanje, jezgro bi krenulo ponovo čim se obrada završi. STATUS pamti done u bistabilu koji se briše tek na sledeći start, jer bi procesor kratak impuls lako propustio.

Mapa je fiksirana u skripti koja gradi sistem, a ne prepuštena automatskom dodeljivanju, tako da rekonstrukcija sistema uvek daje iste adrese.

Master	Slave	Bazna adresa	Opseg
PS M_AXI_GP0	ncc0.S00_AXI (kontrola)	0x5000_0000	4 KB
	ncc0.S01_AXI (memorije)	0x5002_0000	128 KB
	ncc1.S00_AXI	0x5100_0000	4 KB
	ncc1.S01_AXI	0x5102_0000	128 KB
	axi_cdma_0.S_AXI_LITE	0x6000_0000	64 KB
DMA M_AXI	PS S_AXI_HP0 (DDR)	0x0000_0000	iz PS podešavanja
	ncc0.S01_AXI	0x5002_0000	128 KB
	ncc1.S01_AXI	0x5102_0000	128 KB

Tabela 17. Adresna mapa, po masteru

Vidljivost je namerno različita po masteru. DMA ne vidi kontrolne registre, čime se smanjuje šansa da pogrešno programiran prenos upiše nešto u njih; procesor ne vidi DDR kroz programabilnu logiku, jer mu do sopstvene memorije vodi direktan put.

Konstanta modela	Vrednost	Preslikavanje u hardveru
ADDR_NCC	0x5000_0000	ncc0.S00_AXI — identično
ADDR_NCC1	0x5100_0000	ncc1.S00_AXI — identično
ADDR_DMA	0x6000_0000	axi_cdma_0.S_AXI_LITE — identično
ADDR_BRAM	0x4000_0000	ne postoji — nema deljene memorije (odjeljak 8.1)
ADDR_RESULTS	0x40	postaje S01 + 0x10000, dakle drugi interfejs
REG_IMG_ADDR	0x10	<i>rezervisan</i> — bez značenja u slave obrascu
REG_TMP_ADDR	0x14	<i>rezervisan</i>

Tabela 18. Poklapanje sa konstantama sistemskog modela

9.3. Tok podataka po jednom polju table

Softver za svako neprazno polje izvršava sledeći niz. Redosled je određen time što su memorije jednostruko baferovane, pa jezgro ne može računati dok se u njega upisuje. Prenose u svim koracima ispod obavlja procesor, reč po reč — DMA kontroler postoji u topologiji (odjeljak 9.1), ali se ne koristi (odjeljak 10.3).

1. Procesor proverava da li je polje prazno, bez pristupa programabilnoj logici.
2. Procesor u DDR-u priprema bafer sa jednim pikselom po 32-bitnoj reči i izbacuje ga iz keša.
3. Procesor prenosi segment slike u S01 + 0x00000 oba akceleratora.
4. Procesor prenosi šablon u S01 + 0x08000; šabloni se raspakuju jednom na početku programa.
5. Procesor upisuje dimenzije u registre i upisom u CTRL pokreće obradu na oba bloka.
6. Procesor čeka bit done na oba bloka.
7. Procesor prenosi mapu rezultata iz oba bloka u DDR i traži maksimum.

9.4. Dva pravila koja softver mora poštovati

Skorovi se porede kao neoznačeni brojevi (u32), ne kao int32: 0x80000000 ($NCC^2 = 1,0$) je kao označen broj negativan, pa bi traženje maksimuma odbacilo baš savršeno poklapanje. DMA i procesor ne smeju istovremeno pristupati istom S01 interfejsu. Kod istovremenog čitanja i upisa prioritet ima čitanje — upis tada ode na pogrešnu adresu. Sa jednim procesorom i jednim DMA kanalom to se rešava redosledom u softveru (procesor čeka XAxiCdma_IsBusy); hardver to ne proverava sâm.

10. Analiza integrisanog sistema

10.1. Utrošeni resursi

Resurs	Utrošeno	Raspoloživo	Iskorišćenost
Slice LUT	6.225	17.600	35,37 %
Slice registri (FF)	5.020	35.200	14,26 %
Block RAM (ukupno)	39	60	65,00 %
DSP48E1	18	80	22,50 %

Tabela 19. Resursi celog sistema (post-route, xc7z010c[g400-1])

Blok	LUT	FF	RAMB36	DSP
ncc0 (ceo IP)	1.887	1.238	19	9
ncc1 (ceo IP)	1.887	1.238	19	9
axi_interconnect_0	1.622	—	—	—
axi_cdma_0	811	—	—	—

Tabela 20. Raspodela resursa po blokovima

Po instanci je 1.887 LUT, a golo jezgro troši 1.472 (odeljak 7.2); razlika od 415 LUT je cena standardnog interfejsa. Interkonekt i DMA zajedno troše 2.433 LUT.

Blok-memorija je najveće zauzeće i slabost ovog rešenja. Sa 65 % je to jedini resurs bez velike rezerve. Uzrok je 32-bitna reč integralne slike, kod koje je dovoljan 21 bit, jer je najveća moguća suma $90 \cdot 90 \cdot 255 = 2.065.500$. Sužavanje bi oslobodilo oko šest blokova i spustilo zauzeće na približno 55 %. Nije izvedeno jer memorija nije usko grlo, ali je to prva mera ako se doda treća instanca.

10.2. Vremenska analiza i radni takt

Veličina	Vrednost
Radni takt	90,909 MHz (period 11,000 ns)
Najgora vremenska rezerva (WNS)	+0,268 ns
Najgora rezerva za zadržavanje (WHS)	+0,039 ns
Krajnjih tačaka koje krše ograničenje	0 od 15.458
Kritična putanja	ncc0/core_inst/v_reg[2] → ncc0/ms_inst/img_mem/ADDRBWRADDR[14]

Tabela 21. Vremenska analiza sistema (post-route + fizička optimizacija)

Sistem zatvara vremenska ograničenja na 90,909 MHz, sa pozitivnom rezervom i za postavljanje i za zadržavanje, i bez ijedne krajnje tačke koja krši ograničenje.

Traženo je 95 MHz, a PLL procesorskog sistema deli ulaznu frekvenciju celim brojem, pa je najbliža ostvariva vrednost $1000/11 = 90,909$ MHz. Odstupanje od nominalnih 100 MHz iznosi 9,1 %, dakle unutar granice od 20 %.

Ograničenje	WNS	Krajnjih tačaka krši	Takt
11,0 ns (radna tačka)	+0,268 ns	0 od 15.458	90,909 MHz
10,0 ns	+0,032 ns	0 od 15.494	100 MHz

Tabela 22. *Sistem na dva ograničenja takta (post-route + fizička optimizacija)*

Kontrolno merenje pokazuje da sistem **zatvara i 100 MHz**. Radna tačka je ipak zadržana na 90,909 MHz zato što su sva merenja propusnosti izvedena na njoj i zato što je rezerva tamo znatno veća. Izbor niže frekvencije je dakle odluka, a ne tehnička nužnost.

Vezujuća putanja zavisi od ograničenja: na 11 ns to je adresna putanja do memorije slike, a na 10 ns kvadriranje brojioca pred deliocem — ista putanja koja je kritična i u samostalnom jezgru (odjeljak 7.3). Ograničenje je dakle jezgro, a ne integracija: njena cena je razlika između +0,146 ns samostalno i +0,032 ns u sistemu, oko 0,11 ns.

10.3. Propusnost sistema

Ceo tok — učitavanje slike table, petlja 8×8, gruba pretraga, fina potvrda i generisanje FEN notacije — izvršen je **na hardveru**. Rezultat je bit-identičan referentnom modelu: 32 od 32 polja prepoznata, FEN znak po znak jednak zvaničnom.

Faza	Vreme	Udeo
računanje (čekanje jezgra)	1.555 ms	87,3 %
upis u S01	120 ms	6,7 %
čitanje rezultata	67 ms	3,8 %
obrada na procesoru*	40 ms	2,2 %
ukupno	1.782 ms	100 %

Tabela 23. *Izmerena raspodela vremena na ploči (cela tabla, 32 zauzeta polja)*

* 2× smanjivanje segmenta, isecanje segmenta iz slike table i provera praznog polja — sve bez učešća programabilne logike.

Merenje potvrđuje model latencije: model predviđa oko 1.680 ms računanja za 32 polja, pod pretpostavkom da svako polje ima najveći mogući broj kandidata (grubi i fini prolaz). Izmereno je manje, 1.555 ms, jer prazna polja preskaču obradu, a većina zauzetih polja ima svega jedan ili dva kandidata za tip figure, ne maksimalan broj. Obrada iste slike u referentnom rešenju traje 3,667 s, a na ploči je izmereno **1,782 s — 2,06× brže**, uz identičan rezultat.

Prenose u izmerenom toku obavlja procesor, a ne DMA. Razlog je proveren direktno preko JTAG-a, van naše aplikacije: axi_cdma ne uspeva da izvrši prenos duži od dva takta magistrale kroz axi_interconnect_0, konfigurisan kao deljena magistrala (STRATEGY=1) — treći i četvrti beat jednostavno zaglave. Zato prenose radi procesor, reč po reč; iz tabele se vidi da to košta najviše oko 10,5 % ukupnog vremena, jer računanje ionako čini 87 %.

Model latencije omogućava da se izmereno vreme preračuna na drugu veličinu zadatka:

$$T = 2 \cdot \text{img_w} \cdot \text{img_h} + 2 \cdot N + \text{res_w} \cdot \text{res_h} \cdot (N + 112), \quad N = \text{tmp_w} \cdot \text{tmp_h}$$

Veličina zadatka	Taktova	Vreme	Izvor
segment 90×90, šablon 25×15	2.461.201	27,07 ms	izmereno simulacijom
segment 90×90, šablon 30×30	3.783.652	41,6 ms	iz modela

Tabela 24. Propusnost pri radnom taktu od 90,909 MHz

Pošto u sistemu rade dve instance paralelno, sistem obrađuje dva polja table u vremenu u kome bi jedna instanca obradila jedno.

10.4. Poređenje sa referentnim rešenjem

Veličina	Referentno rešenje	Ovaj rad
LUT po instanci	5.269	1.887
DSP po instanci	16	9
Blok-memorija po instanci	4 manja bloka	19 većih ← slabije
LUT za dve instance	10.538 (59,9 %)	ceo sistem 6.225 (35,4 %)
Taktova za 90×90 / 30×30	10.360.183	3.783.652

Tabela 25. Poređenje sa rešenjem iz visokonivoske sinteze

Ceo sistem — dve instance, interkonekt i DMA zajedno — staje u manje LUT-ova nego što referentno rešenje troši samo na svoja dva jezgra. Po broju taktova na istom zadatku razlika je 2,74×. Jedina veličina po kojoj je ovo rešenje slabije je blok-memorija, sa uzrokom navedenim u odeljku 10.1.

11. Zaključak

RT model NCC jezgra je opisan u VHDL-u, funkcionalno verifikovan, sintetizovan i implementiran za uređaj xc7z010c1g400-1, zatim spakovan u IP blok sa dva AXI interfejsa i integrisan u sistem sa procesorskim sistemom Zynq, drugom instancom akceleratora i DMA kontrolerom. Ključni rezultati:

- rezultat je **bit-identičan** referentnom modelu u jeziku C, i ostaje takav i kroz AXI omotač (odeljak 7.5);
- samostalno jezgro zatvara vremenska ograničenja i pri 100 MHz, sa rezervom od 0,146 ns i ostvarivom frekvencijom od približno 101,5 MHz (odeljak 7.3);
- latencija po pozivu iznosi 2.461.201 takt, odnosno 1,31 takta po piksel-operaciji; pri 90,909 MHz to je 27,07 ms i približno 185.300 pozicija prozora u sekundi (odeljak 7.4);
- ceo sistem zauzima **35,4 % LUT-ova** uređaja, manje nego što referentno rešenje troši samo na svoja dva jezgra (59,9 %), i po instanci 1.887 naspram 5.269 LUT (odeljci 10.1 i 10.4);
- sistem zatvara vremenska ograničenja na 90,909 MHz, sa rezervom od +0,268 ns i bez ijedne od 15.458 krajnjih tačaka koja krši ograničenje (odeljak 10.2);
- na istom zadatku sistem troši **2,74× manje taktova** od referentnog rešenja: 3.783.652 naspram 10.360.183 (odeljak 10.4).

Slabost se navodi otvoreno: blok-memorija je jedini resurs bez rezerve, sa 65 % zauzeća, i jedina veličina po kojoj je ovo rešenje slabije od referentnog. Uzrok je 32-bitna reč integralne slike kod koje je dovoljan 21 bit; suženje bi zauzeće spustilo na oko 55 % (odeljak 10.1).

Radni takt nije ograničenje: sistem zatvara i 100 MHz (rezerva +0,032 ns). Radna tačka je zadržana na 90,909 MHz zato što su sva merenja izvedena na njoj i zato što je rezerva tamo veća; odstupanje od 9,1 % je unutar dopuštenih 20 % (odeljak 10.2).

Sistem je proveren na realnoj ploči. Konfiguraciona datoteka je generisana, softver prenesen iz izvršne specifikacije, i ceo tok izvršen na hardveru: 32 od 32 polja prepoznata, FEN znak po znak jednak zvaničnom, ukupno 1,782 s naspram 3,667 s referentnog rešenja (odeljak 10.3).

12. Funkcionalna verifikacija

12.1. Uvod i nivoi verifikacije

Predmet verifikacije je IP jezgro `ncc_accel` — hardverski akcelerator normalizovane unakrsne korelacije, projektovan i implementiran u okviru predmeta Projektovanje složenih digitalnih sistema. Jezgro je opisano u VHDL-u i verifikovano na nivou RTL-a golden podacima; ovo poglavlje opisuje izgradnju verifikacionog okruženja u SystemVerilog/UVM metodologiji nad istim, nepromenjenim projektom.

Računsko jezgro `ncc_core` već je provereno na nivou RTL-a (golden skup $4 \times 4/2 \times 2$ i realni segment 90×90 , bit-tačno), pa se težište ovde pomera na **IP nivo** — na `ncc_accel` sa dva magistralna interfejsa: AXI4-Lite (S00, kontrolni registri) i AXI4-Full (S01, memorijski prostor). Integracioni nivo (`ncc_system`: dve instance jezgra i AXI interconnect) razmatra se tek po stabilizaciji okruženja na IP nivou.

12.2. Granice dopuštenog stimulusa

Dok je u stanju `S_IDLE`, modul `ncc_core` na uzlaznu ivicu signala start proverava četiri uslova nad dimenzijama. Provere su napisane kao VHDL tvrdnje sa nivoom ozbiljnosti `failure`. Ta četiri uslova mora da poštuje svaki softver koji upravlja jezgrom, pa i verifikaciono okruženje.

Oznaka	Uslov	Poruka pri kršenju
U1	$1 \leq \text{img_w} \leq 90$ i $1 \leq \text{img_h} \leq 90$	<code>img_w/img_h</code> van opsega 1..90
U2	$1 \leq \text{tmp_w} \leq 30$ i $1 \leq \text{tmp_h} \leq 30$	<code>tmp_w/tmp_h</code> van opsega 1..30
U3	$\text{tmp_w} \cdot \text{tmp_h} \leq 900$	$\text{tmp_w} * \text{tmp_h} > \text{MAX_TMP_PIX}$
U4	$\text{tmp_w} \leq \text{img_w}$ i $\text{tmp_h} \leq \text{img_h}$	sablon veci od slike

Tabela 26. Uslovi nad dimenzijama koje jezgro proverava pri pokretanju

Za verifikaciju to nije formalnost. Nivo ozbiljnosti `failure` obara **celu simulaciju**, a ne pojedinačni test. Iz toga slede tri pravila:

- Ograničenja randomizacije u klasi sekvencijalne stavke moraju garantovati U1–U4. Slučajno generisana dimenzija van opsega ne daje neuspeh testa nego rušenje regresije, čime se gubi i pokrivenost svih ostalih testova iz tog pokretanja.
- Ponašanje pri *nelegalnim* dimenzijama namerno se ne proverava. Tvrdnje žive samo u simulaciji — sinteza ih izbacuje, pa bi u hardveru prevelika dimenzija prošla kao tiho odsecanje bita. Proverava se, dakle, da li se ugovor poštuje, a ne šta se dešava kada se prekrši.
- Registri su 32-bitni, a jezgro čita samo donjih osam bita. Gornja 24 bita mogu se upisati i pročitati nazad, ali na rad jezgra ne utiču. Takav upis je sasvim legalan i zgodan za proveru „upisano jednako pročitano“, nezavisno od toga šta jezgro računa.

Signal `img_w` je tipa `unsigned(7 downto 0)`, pa sa magistrale prima bilo koju vrednost od 0 do 255. Vrednosti od 91 do 255 fizički se mogu upisati u registar, ali ih jezgro odbija tvrdnjom U1. Granicu, dakle, ne postavlja širina registra nego ugovor.

12.3. Kontrolni registri (AXI4-Lite, S00)

Adresna magistrala interfejsa S00 široka je šest bita ($C_S00_AXI_ADDR_WIDTH = 6$), uz $ADDR_LSB = 2$ i $OPT_MEM_ADDR_BITS = 3$, što daje šesnaest registara u opsegu 0x00–0x3C.

Adresa	Registar	Naziv	Ponašanje
0x00	slv_reg0	REG_IMG_W	bitovi 7:0 vode na <code>img_w</code>
0x04	slv_reg1	REG_IMG_H	bitovi 7:0 vode na <code>img_h</code>
0x08	slv_reg2	REG_TMP_W	bitovi 7:0 vode na <code>tmp_w</code>
0x0C	slv_reg3	REG_TMP_H	bitovi 7:0 vode na <code>tmp_h</code>
0x30	slv_reg12	REG_CTRL	upis sa <code>WDATA(0)='1'</code> daje impuls <code>start</code> i briše <code>done_sticky</code>
0x34	slv_reg13	REG_STATUS	bit 0 = <code>done_sticky</code> , bit 1 = <code>busy</code> ; samo za čitanje

Tabela 27. Registri interfejsa S00 koje jezgro stvarno koristi

Pokretanje jezgra tokom rada nije ni na koji način zaštićeno. Uslov za generisanje impulsa `start` glasi `wr_beat='1'` and `mem_logic="1100"` and `WDATA(0)='1'`, bez provere signala `busy`. Upis u `REG_CTRL` dok jezgro radi zato istovremeno generiše impuls koji `ncc_core` zanemaruje (jer nije u stanju `S_IDLE`) i **briše zastavicu** `done_sticky`. Reč je o stvarnom, dosad neproverenom ponašanju i o prioritarnoj meti plana (test T7 u tabeli 29).

12.4. Memorijski prostor (AXI4-Full, S01)

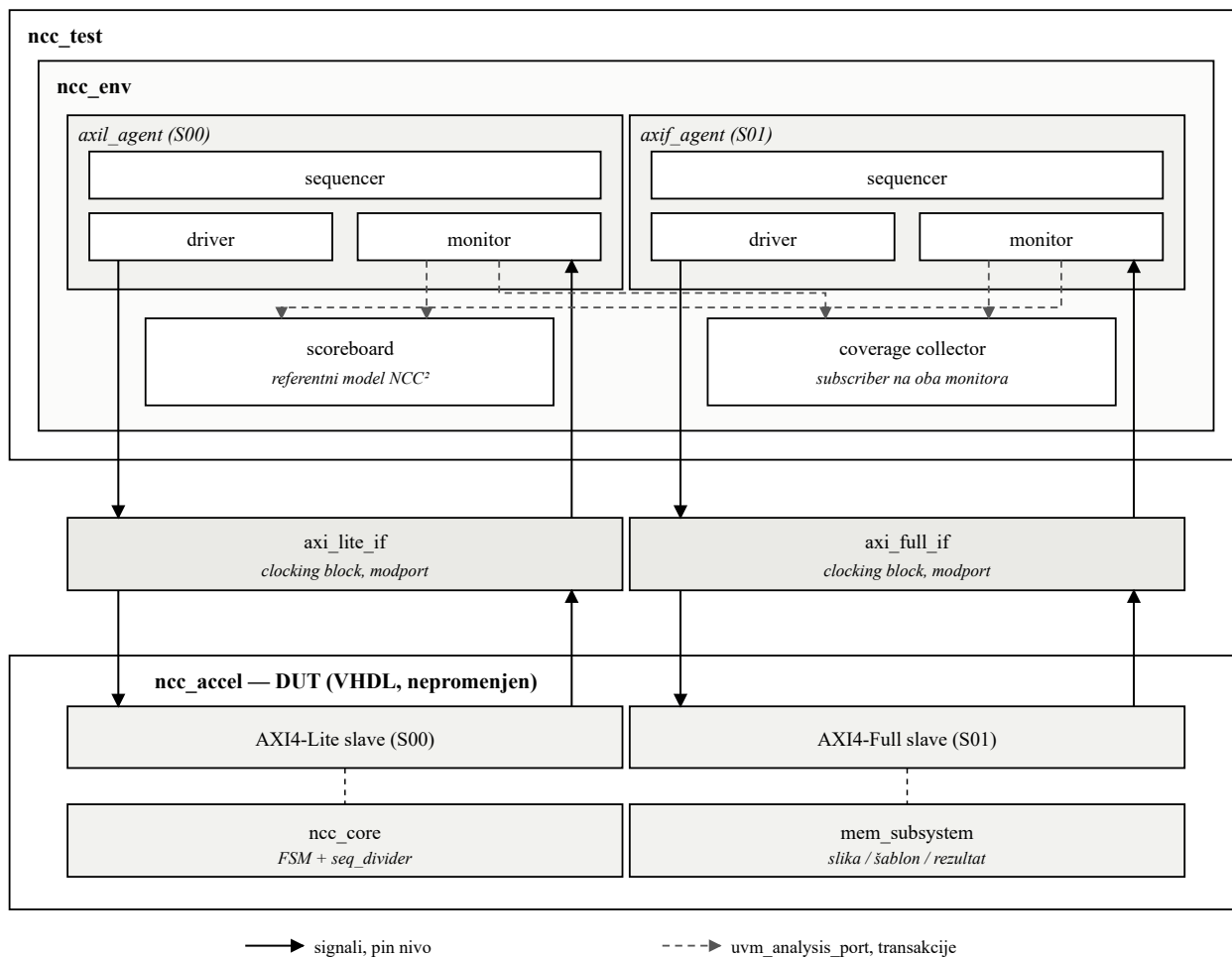
Adresna magistrala interfejsa S01 široka je sedamnaest bita, što daje prostor od 128 KB. Modul `mem_subsystem` dekodira region iz bitova 16:15, a adresu reči iz bitova 14:2 — dakle jedan piksel po 32-bitnoj reči.

Region	Bitovi 16:15	Pomeraj	Memorija	U upotrebi	Pristup
Slika	00	+0x00000	8192×8	8100	čitanje i upis
Šablon	01	+0x08000	1024×8	900	čitanje i upis
Rezultat	10	+0x10000	8192×32	8100	samo čitanje
Nemapirano	11	+0x18000	—	—	čita nule

Tabela 28. Regioni memorijskog prostora S01

12.5. Struktura verifikacionog okruženja

Okruženje prati uobičajenu UVM hijerarhiju. Test na vrhu gradi okruženje, a okruženje sadrži dva agenta — po jedan za svaki magistralni interfejs — zajedno sa komponentom za proveru rezultata i kolektorom pokrivenosti. Oba agenta imaju istu unutrašnju građu, pa se razlikuju samo po protokolu koji opslužuju.



Slika 7. Hijerarhija verifikacionog okruženja. Drajveri pobuđuju DUT kroz virtualne interfejsa, dok monitori isti saobraćaj pasivno rekonstruišu i prosleđuju komponenti za proveru i kolektoru pokrivenosti transakcionim putem.

Ključna osobina ovakve podele jeste da komponenta za proveru ne prima ništa neposredno od pobude. Sve što zna stiže preko monitora, pa se isto okruženje može upotrebiti i nad dvema instancama jezgra u integrisanom sistemu, bez izmena.

12.6. Zahtev na drajver izveden iz izvornog koda

Upisni automat interfejsa S00 strogo je dvofazan. U stanju `waddr` spreman je samo adresni kanal (`AWREADY='1'`, `WREADY='0'`), a tek po prihvatanju adrese, u stanju `wdata`, spreman je kanal podataka (`AWREADY='0'`, `WREADY='1'`). Ta dva signala nikada se ne postavljaju u istom taktu.

Takav raspored nije slučaj — njime je ispravljena ranije otkrivena protokolska greška, zbog koje je slave prihvatao podatak pre nego što je primio adresu. Odatle sledi obavezan zahtev na drajver: kanali `AW`, `W` i `B` moraju se voditi kao tri nezavisne niti, uz zadržavanje signala `WVALID` sve dok ne stigne `WREADY`. Čeka li drajver na oba signala odjednom, blokira se zauvek; to se i praktično videlo pri prvoj probi mešovitog jezika, kada je test prekinuo tek nadzorni brojač. Podrška za redosled u kome podatak prethodi adresi zato nije stvar stila nego uslov da taj scenario uopšte prođe.

12.7. Matrica testova

Oznaka **D** označava usmereni test, a **R** test sa ograničenom slučajnom pobudom.

Br.	Funkcija	Tip	Konkretna pobuda	Cilj pokrivenosti
T1	Upis i čitanje registara S00	D+R	svih šest korišćenih registara; slučajna vrednost	svaka adresa, oba smera
T2	Neupotrebljeni registri	R	slv_reg4–11, 14, 15	bin „scratch“
T3	Redosled AW/W na S00	D+R	tri scenarija; raspodela 2 : 2 : 1	ukrštanje redosled × region
T4	Redosled AW/W na S01	D+R	ista tri scenarija	ukrštanje redosled × region
T5	Paketni prenos na S01	D+R	dužine 1, 2, 3, 16 i 256; WSTRB 0xF, 0x1, 0x3, 0x9	bins po dužini paketa
T6	Sled start – busy – done	D	90×90 / 25×15, čitanje statusa u tri trenutka	ukrštanje start × busy
T7	Pokretanje tokom rada	D	upis u REG_CTRL dok je busy= '1'	bin (start=1, busy=1)
T8	Rezultat, veliki golden skup	D	90×90 / 25×15; vrh 0x80000000 na (32, 14)	—
T9	Rezultat, mali golden skup	D	4×4 / 2×2, svih devet pozicija	—
T10	Rezultat, slučajne dimenzije	R	slika 8–90, šablon 2–30, uz U3 i U4	bins dimenzija
T11	Šablon jednak slici	D	30×30 / 30×30, mapa rezultata 1×1	bin najmanje mape
T12	Najveći dopušteni šablon	D	90×90 / 30×30, tačno 900 piksela	bin najvećeg šablona
T13	Region rezultata je samo za čitanje	D	upis na +0x10000, pa čitanje	bin upisa u region 10
T14	Aliasiranje šablona	D	upis na +0x08000 i +0x09000	bin aliasa
T15	Nemapirani region	D	čitanje sa +0x18000	bin regiona 11

Tabela 29. Matrica funkcija za verifikaciju

Regresija pokreće svaki slučajni test sa dvadeset različitih početnih vrednosti generatora; usmereni testovi idu sa jednom, jer im slučajnost ne menja pobudu. Testovi T8 i T10 izvode pun proračun i traju minutima po pokretanju — oko 2,4 miliona taktova — pa se drže u zasebnoj, sporoj listi. Brza lista, koja obuhvata preostale testove, mora da se izvrši u sekundama, jer se jedino tako može pokretati posle svake izmene.

12.8. Plan pokrivenosti

Pokrivenost se uzorkuje isključivo iz monitora, nikada iz drajvera. Uzorkovanje iz drajvera merilo bi svojstva pobude, a ne ponašanje projekta, i davalo bi lažno visoke brojeve.

```

covergroup cg_dimenzije
  img_w : bins {1, [2:44], 45, [46:89], 90}
  img_h : isti raspored
  tmp_w : bins {1, [2:14], 15, [16:29], 30}
  tmp_h : isti raspored
  cross img_w x tmp_w    uz ignore_bins (tmp_w > img_w)    -- U4 ih cini nedostiznim

covergroup cg_kontrola
  start : bins {0, 1}      busy : bins {0, 1}      done : bins {0, 1}
  cross start x busy      -- bin (1,1) je test T7
  cross start x done      -- pokretanje brise zastavicu done_sticky

covergroup cg_axi
  redosled : bins {AW_FIRST, W_FIRST, SIMUL}
  region   : bins {S00, S01_slika, S01_sablon, S01_rezultat, S01_nemapirano}
  cross redosled x region -- petnaest kombinacija, sredisnji cilj plana
  burst_len : bins {1, 2, [3:15], [16:255], 256}
  wstrb      : bins {0xF, 0x1, 0x3, 0x9}

```

Cilj je potpuna pokrivenost ukrštanja redosleda i regiona, jer upravo to ukrštanje pogađa ispravljenu protokolsku grešku; ukupna pokrivenost treba da bude najmanje 90 %. Uz svaki nepokriven bin mora stajati razlog — nepokrivenost bez objašnjenja čita se kao propust, a ne kao odluka.

12.9. Realizovano okruženje

U ovom odeljku opisano je okruženje koje je napravljeno na osnovu plana iz prethodnih odeljaka. Tabela 30 daje pregled izvornih datoteka i njihovog sadržaja.

Datoteka	Sadržaj
src/ncc_verif_pkg.sv	zajednički tipovi: redosled AW/W, regioni S01, adrese registara
src/interfaces/axi_lite_if.sv, axi_full_if.sv	interfejsi sa blokovima takta za drajver i monitor; SVA protokolske tvrdnje
src/axil_agent/*.sv	agent S00: ncc_reg_item, sekvencer, drajver, monitor, konfiguracija
src/axif_agent/*.sv	agent S01: ncc_mem_item, paketni drajver, monitor, konfiguracija
src/ref_model/ncc_ref_pkg.sv	bihevioralni referentni model NCC ² , bit-tačan, bez DPI-a
src/env/*.sv, src/sequences/*.sv	ncc_env, virtuelni sekvencer, ncc_scoreboard, ncc_coverage, sekvence
tests/*.sv	devet testova, od provere interfejsa do slučajnog testa
tb/tb_top.sv	takt, reset, instanca DUT-a, prosleđivanje interfejsa kroz uvm_config_db
sim/*.sh	common.sh, build.sh, run_one.sh, regress.sh, coverage.sh i njihovi parovi za xrun

Tabela 30. Izvorne datoteke verifikacionog okruženja

Pored komponenata sa slike 7, u oba interfejsa stoje SVA tvrdnje: stabilnost VALID signala i sadržaja do READY na svim kanalima, BVALID tek posle WLAST, i invarijanta !(awready && wready). Poslednja je ispravka AW/W greške iz odeljka 12.6 zapisana u izvršnom obliku: ako se oba signala ikada podignu u

istom taktu, greška se vratila. Pošto su tvrdnje u interfejsima, važe u svakom testu, i u onima koji o njima ne znaju ništa.

Scoreboard sluša isključivo monitore. Od onoga što je viđeno na magistrali gradi senku memorija slike i šablona, na upis u REG_CTRL poziva referentni model i poredi svaku pročitane reč regiona rezultata sa predikcijom. Ako na kraju testa nije izvršio nijedno poređenje, prijavljuje grešku. Scoreboard koji ništa nije uporedio nije dokaz ispravnosti.

12.10. Rezultati regresije (Vivado XSim 2025.2)

Regresija se pokreće jednom komandom, `./sim/regress.sh` sve. Elaboracija se radi jednom za celu regresiju, a svaki test se zatim pokreće iz istog snimka. Test prolazi ako se završi bez ijedne greške (UVM_ERROR, UVM_FATAL, pad SVA tvrdnje) i ako log sadrži završni UVM izveštaj. Druga provera postoji zato što bez nje skripta ne razlikuje test bez grešaka od testa koji se uopšte nije pokrenuo.

Test	Seed	Slika / šablon	Sim. vreme	Poređenja	Pokr.	Trajanje	Ishod
<i>brza lista</i>							
ncc_vif_test	1	—	0,3 µs	—	—	3 s	PASS
ncc_axil_smoke_test	1	—	0,5 µs	7*	—	3 s	PASS
ncc_axil_agent_test	1	—	0,5 µs	14*	—	3 s	PASS
ncc_axif_smoke_test	1	—	1,3 µs	6*	—	3 s	PASS
ncc_smoke_test	1	4×4 / 2×2	11,8 µs	9	56,0 %	3 s	PASS
<i>spora lista</i>							
ncc_cov_test	1	8 skupova	765,5 µs	8	95,1 %	40 s	PASS
ncc_dim_sweep_test	1	19 skupova	875,6 µs	8	76,0 %	44 s	PASS
<i>slučajna lista</i>							
ncc_random_test	1	16×18 / 1×2	323,5 µs	272	61,4 %	18 s	PASS
ncc_random_test	2	5×11 / 4×1	29,1 µs	22	57,8 %	5 s	PASS
ncc_random_test	3	18×3 / 2×3	23,6 µs	17	59,6 %	4 s	PASS
ncc_random_test	4	21×13 / 2×4	253,8 µs	200	57,8 %	14 s	PASS
ncc_random_test	5	8×18 / 2×3	141,2 µs	112	58,2 %	10 s	PASS
ncc_random_test	6	13×10 / 4×1	123,7 µs	100	58,2 %	8 s	PASS
ncc_random_test	7	19×12 / 3×3	217,7 µs	170	60,0 %	13 s	PASS
ncc_random_test	8	6×5 / 6×4	6,7 µs	2	59,1 %	4 s	PASS
ncc_random_test	9	18×18 / 5×1	311,7 µs	252	59,1 %	17 s	PASS
ncc_random_test	10	18×6 / 1×3	89,3 µs	72	58,7 %	7 s	PASS
ncc_random_test	11	17×14 / 5×3	211,0 µs	156	57,8 %	13 s	PASS
ncc_random_test	12	13×18 / 6×3	178,0 µs	128	58,7 %	11 s	PASS
ncc_random_test	13	7×23 / 5×3	88,7 µs	63	58,7 %	7 s	PASS
ncc_random_test	14	11×18 / 1×5	191,4 µs	154	60,0 %	12 s	PASS
ncc_random_test	15	13×14 / 6×2	139,2 µs	104	58,2 %	9 s	PASS
ncc_random_test	16	21×23 / 4×3	493,4 µs	378	59,6 %	26 s	PASS
ncc_random_test	17	6×16 / 2×4	84,0 µs	65	58,2 %	7 s	PASS
ncc_random_test	18	18×23 / 6×1	373,2 µs	299	57,8 %	20 s	PASS
ncc_random_test	19	17×9 / 3×5	103,5 µs	75	55,1 %	8 s	PASS
ncc_random_test	20	22×24 / 3×3	559,6 µs	440	59,6 %	29 s	PASS
Ukupno: 27 pokretanja				3106	100 %	341 s	27 / 0

Tabela 31. Puna regresija, 5. septembar 2026, 14:33

* Testovi agenata nemaju scoreboard. Broj označava provere „upisano jednako pročitano“ (S00, S01), odnosno transakcije koje je monitor rekonstruisao (ncc_axil_agent_test). Kolona „Pokr.“ je pokrivenost jednog pokretanja kako je javlja ncc_coverage; zbirni red je spojena pokrivenost svih 23 baze (odeljak 12.11).

Svih 27 pokretanja završilo se sa nula upozorenja, nula grešaka i nula fatalnih poruka. Scoreboard je ukupno uporedio **3106 vrednosti mape rezultata** sa referentnim modelom i nije našao nijedno neslaganje. Najveći deo tih poređenja (3081) potiče iz slučajnog testa, u kome se svaki seed završava potpunim čitanjem mape. Ispis jednog pokretanja izgleda ovako (ncc_smoke_test):

```
@ 345000: [SMOKE] slika i sablon upisani u S01
@ 515000: [SB] predikcija spremna: slika 4x4, sablon 2x2, mapa 9 vrednosti
@ 515000: [SMOKE] CTRL.start postavljen
@ 11715000: [SMOKE] done_sticky postavljen
@ 11815000: [SMOKE] rezultat[0] (u=0, v=0) = 0x7fd2e248
@ 11815000: [SMOKE] rezultat[4] (u=1, v=1) = 0x80000000
@ 11815000: [SMOKE] rezultat[8] (u=2, v=2) = 0x603b88fe
@ 11815000: [SMOKE] ceo put kroz DUT prosao: S01 -> S00 -> start -> done -> rezultat
@ 11815000: [SB] 9 poredjenja, 0 neslaganja
@ 11815000: [COV] pokrivenost: dimenzije 17.1%, kontrola 90.0%, AXI 61.0%, ukupno 56.0%
--- UVM Report Summary ---
UVM_INFO : 22 UVM_WARNING : 0 UVM_ERROR : 0 UVM_FATAL : 0
```

Šablon u ovom testu je doslovno prozor slike na poziciji (1, 1), pa tamo mora stajati tačno 0x80000000 ($NCC^2 = 1$, u formatu Q1.31), a na svih ostalih osam pozicija strogo manje.

Test agenta S00 izmerio je redosled na samoj magistrali, po trenutku postavljanja VALID signala: AW_FIRST=2, W_FIRST=3, SIMUL=2, tačno onako kako je sekvenca zadala. Test agenta S01 proverio je pakete dužine 1, 2, 3 i 16, a ncc_cov_test i dužinu od 256, uz sva tri redosleda.

12.11. Funkcionalna pokrivenost

Pokrivenost se meri preko cele regresije, a ne po testu. Svako pokretanje ostavlja svoju bazu, a ./sim/coverage.sh ih spaja alatom xcrng. U spajanju su 23 baze: ncc_smoke_test, obe spore i 20 slučajnih. Testovi agenata ne instanciraju ncc_env i zato nemaju bazu. Zaglavlje izveštaja navodi „Number of Tests: 1“, jer xcrng spojene baze broji kao jedno pokretanje.

Grupa	Tačka / ukrštanje	Bin-ova	Pokriveno	%
cg_dimenzije	cp_img_w	5	5	100
	cp_img_h	5	5	100
	cp_tmp_w	5	5	100
	cp_tmp_h	5	5	100
	x_img_tmp (bez nedostižnih)	19	19	100
cg_kontrola	cp_start	2	2	100
	cp_busy	2	2	100
	cp_done	2	2	100
	x_start_busy	4	4	100
	x_start_done	4	4	100
cg_axi	cp_order	3	3	100
	cp_region	5	5	100
	cp_len	5	5	100
	cp_strb	4	4	100
	x_order_region	15	15	100
Ukupno (Coverage Score)		90	90	100

Tabela 32. *Spojena pokrivenost po tačkama*

Središnji cilj plana bilo je ukrštanje redosleda AW/W i regiona, jer ono pogađa ispravljenu protokolsku grešku. Tabela 33 daje broj pogodaka po svakom od petnaest bin-ova.

Redosled	S00	S01 slika	S01 šablon	S01 rezultat	S01 nemapirano	Ukupno
AW_FIRST	109	49	56	28	39	281
W_FIRST	114	17	10	1	1	143
SIMUL	98	12	5	1	1	117

Tabela 33. *Ukrštanje x_order_region, broj pogodaka po bin-u*

12.12. Dvostruka simulacija: Vivado XSim i Cadence Xcelium

Ista regresija, sa istim spiskom testova, istim redosledom i istom proverom PASS/FAIL, pokrenuta je na oba simulatora. Skripta regress.sh prima SIM=xsim ili SIM=xrun; razlikuje se samo koji par skripti za elaboraciju i pokretanje se poziva.

	Vivado XSim 2025.2	Cadence Xcelium 19.03
Testovi	27 prošlo, 0 palo	27 prošlo, 0 palo
Trajanje regresije	341 s	67 s
Elaboracija	xvhd1 -2008, xvlog -sv, xelab	xrun -elaborate -v200x -relax -uvm
UVM	1.2 (uz Vivado)	CDNS-1.1d (uz Xcelium)

Tabela 34. *Ishod regresije na dva simulatora*

Isto okruženje radi na oba simulatora bez izmena u SystemVerilog kodu i sa istim ishodom, a Xcelium je regresiju izvršio oko pet puta brže.

12.13. Nalazi verifikacije

Nalazi su podeljeni po tome gde se greška nalazila: u DUT-u ili u okruženju.

Br.	Nalaz	Gde	Kako je otkriven	Test / provera
N1	Rezultati se čitaju samo preko S01	DUT (S00)	čitanje RTL-a	odeljak 12.3
N2	Start dok je jezgro zauzeto briše done_sticky	DUT (S00)	čitanje RTL-a	T7, bin (1,1)
N3	Ponašanje memorijskog interfejsa S01	DUT (S01)	čitanje RTL-a	T13, T14, T15
N4	Paket duži od 256 beat-ova	okruženje (pobuda)	regresija, 6 od 20 seed-ova	ncc_random_test

Tabela 35. Pregled nalaza

N1 — Registarska mapa

Adresni prostor interfejsa S00 širok je šest bita, pa je najviša adresa registra 0x3C. Mapa rezultata ne čita se preko kontrolnih registara, nego isključivo preko S01, iz regiona rezultata (+0x10000). Registri slv_reg4 i slv_reg5 (0x10 i 0x14) mogu da se upišu i pročitaju, ali nisu povezani sa jezgrom i ne utiču na njegov rad.

N2 — Pokretanje tokom rada

Upis u REG_CTRL dok jezgro radi nije blokiran. Generiše se impuls start, koji jezgro zanemaruje jer nije u stanju S_IDLE, ali se pri tome briše zastavica done_sticky. Softver zato ne sme da upisuje u CTRL pre nego što pročita da je prethodna obrada završena. Scenario je proveren u testu ncc_cov_test na skupu 60×2 / 20×2: posle ponovnog upisa dok je busy=1, jezgro je normalno završilo obradu i postavilo done_sticky.

N3 — Ponašanje memorijskog interfejsa S01

Interfejs S01 ima četiri osobine koje ne zavise od samog NCC proračuna. Region rezultata je samo za čitanje: upis se zanemaruje, a odgovor na magistrali je ipak OKAY. Memorija šablona koristi samo donjih deset bita adrese reči, pa se adrese koje se razlikuju za 1024 reči preslikavaju na istu lokaciju. Pri čitanju slike i šablona gornja 24 bita su nule, dok region rezultata vraća punu 32-bitnu reč. Nemapirani region vraća nule umesto odgovora DECERR.

Nule u gornja 24 bita i čitanje nemapiranog regiona proveravaju se eksplicitno, dok su upis u region rezultata i preslikavanje adresa šablona pobuđeni u testu pokrivenosti. Proveren je i signal WSTRB: upis sa spuštenim bitom 0 ne menja memoriju, jer je mem_we_o <= wr_beat and S_AXI_WSTRB(0).

N4 — Granica od 256 beat-ova

Slučajni test je u prvoj verziji padao za šest od dvadeset seed-ova. Sekvenca je celu sliku slala kao jedan AXI paket, a paket može imati najviše 256 beat-ova; svi seed-ovi koji su pali imali su sliku veću od 256 piksela. Greška je bila u pobudi, ne u DUT-u, i ispravljena je deljenjem prenosa na pakete koji poštuju

granicu od 256 beat-ova i granicu od 4 KB. Ovaj nalaz pokazuje zašto se slučajni test pokreće sa više seed-ova: sa jednim, manjim ulazom greška ne bi bila primećena.

12.14. Zaključak verifikacije

Verifikacija `ncc_accel` zatvorena je na IP nivou. Svih 27 pokretanja regresije prolazi na oba simulatora, Vivado XSim i Cadence Xcelium; u XSim-u bez ijednog UVM upozorenja. Scoreboard je uporedio 3106 vrednosti rezultata sa referentnim modelom bez ijednog neslaganja. Funkcionalna pokrivenost iznosi 100 % u sve tri grupe, uključujući ukrštanje redosleda AW/W i regiona, koje cilja ranije ispravljenu protokolsku grešku, a SVA tvrdnja u interfejsu čuva da se ta greška ne vrati.

Pored potvrde da DUT radi ispravno, verifikacija je dala četiri konkretna nalaza. Tri se odnose na ponašanje DUT-a koje softver mora da poštuje: raspored registara i memorije, brisanje zastavice `done_sticky` pri ponovnom pokretanju i osobine memorijskog interfejsa S01. Četvrti je greška u pobudi, koju je otkrila regresija sa više seed-ova.